"The Essentials of Computer Organization and Architecture"

Unit 5 – Exercises and Solutions

EAT1 A computational system with a 3-level memory hierarchy has average access times of 20 ns (level 1), 200 ns (level 2) and 2 ms (level 3). For level 1 and 2, the average hit rates are 90% and 95%.
a) Compute the effective (average) access time of this memory hierarchy.

EAT1 A computational system with a 3-level memory hierarchy has average access times of 20 ns (level 1), 200 ns (level 2) and 2 ms (level 3). For level 1 and 2, the average hit rates are 90% and 95%.
a) Compute the effective (average) access time of this memory hierarchy.

Solution a):

- *H1* = 0,90; *H2* = 0,95
- *T1* = 20 ns ; *T2* = 200 ns ; *T3* = **2 ms** = **2 x 10^6 ns**

- **overlapped access**:
  $EAT_3 = H1xT1 + (1-H1) \times [\ H2xT2 + (1-H2)xT3\ ]$
  = 0,9 x 20 + (1-0,9) x [0,95 x 200 + (1-0,95) x 2x10^6]
  = 18 + 0,1 x [190 + 100000]
  = 18 + 10019 = 10037 ns = 10,037 us = 0,010037 ms

- **sequential access**:
  $EAT_3 = H1xT1 + (1-H1) \times [\ H2 \times (T1+T2\ ) + (1-H2) \times (T1+T2+T3)\ ]$
  = 0,9 x 20 + (1-0,9) x [0,95 x (20+200) + (1-0,95) x (20+200+2x10^6)]
  = 18 + 0,1 x [209 + 100011]
  = 18 + 10022 = 10040 ns = 10,040 us = 0,010040 ms

EAT1 A computational system with a 3-level memory hierarchy has average access times of 20 ns (level 1), 200 ns (level 2) and 2 ms (level 3). For level 1 and 2, the average hit rates are 90% and 95%.
b) Compute the probability of having to access levels 2 and 3 (separately).

EAT1 A computational system with a 3-level memory hierarchy has average access times of 20 ns (level 1), 200 ns (level 2) and 2 ms (level 3). For level 1 and 2, the average hit rates are 90% and 95%. b) Compute the probability of having to access levels 2 and 3 (separately).

Solution b):

- *H1* = 0,90; *H2* = 0,95
- **overlapped access**:
  $EAT_3 = H1xT1 + (1-H1) \times [\, H2xT2 + (1-H2)xT3 \,]$
- **sequential access**:
  $EAT_3 = H1xT1 + (1-H1) \times [\, H2 \times (T1+T2) + (1-H2) \times (T1+T2+T3) \,]$

P1 = H1= 0,90
(probability of having to acesso level 1 only)
P2 = (1-H1) x H2 = (1-0,9) x 0,95 = 0,095
(probability of having to access level 2)
P3 = (1-H1) x (1-H2) = (1-0,9) x (1-0,95) = 0,005
(probability of having to access level 3)

verification: P1+P2+P3 = 0,9 + 0,095 + 0,005 = 1

EAT2 A system with a 4-level memory hierarchy has average access times of 1 ns (level 1), 10 ns (level 2), 100 ns (level 3) and 1 us (level 4). For the first 3 levels, the average hit rates are 90%, 94% and 97%. Compute the average access time of this hierarchy.

EAT2 A system with a 4-level memory hierarchy has average access times of 1 ns (level 1), 10 ns (level 2), 100 ns (level 3) and 1 us (level 4). For the first 3 levels, the average hit rates are 90%, 94% and 97%. Compute the average access time of this hierarchy.

Solution:

- *H1* = 0,90; *H2* = 0,94; *H3* = 0,97;
- *T1* = 1 ns ; *T2* = 10 ns ; *T3* = 100 ns ; *T4* = **1 us = 1 x 10^3 = 1000 ns**

- **overlapped access**:

  $EAT_4 = H1xT1 + (1-H1) \times [\, H2xT2 + (1-H2) \times [\, H3xT3 + (1-H3)xT4 \,] \,]$

  = 0,9x1 + (1-0,9) x [0,94x10 + (1-0,94) x [0,97x100 + (1-0,97)x1000]]

  = 2,6 ns

- **sequential access**:

  $EAT_4 = H1xT1 + (1-H1) \times [\, H2x(T1+T2) + (1-H2) \times [\, H3x(T1+T2+T3) + (1-H3)x(T1+T2+T3+T4)] \,]$

  = 0,9x1 + (1-0,9) x [0,94x(1+10) + (1-0,94) x [0,97x(1+10+100) + (1-0,97)x(1+10+100+1000)]]

  = 2,8 ns

EAT3 a) Deduce the EAT expression for a 5-level memory hierarchy. b) Apply it reusing the times and probabilities of exercise EAT2, and considering a hit-rate in level 4 of 99% and access time of 1 ms in level 5.

EAT3 a) Deduce the EAT expression for a 5-level memory hierarchy. b) Apply it reusing the times and probabilities of exercise EAT2, and considering a hit-rate in level 4 of 99% and access time of 1 ms in level 5.

Solution:

a) In the 4-level EAT expression, we must adapt the **terminal** component:

- **overlapped access**:

$$EAT_5 = H1xT1 + (1-H1)\ x\ [\ H2xT2 + (1-H2)\ x\ [\ H3xT3 + (1-H3)x(*)\ ]\ ]$$

$$EAT_5 = H1xT1 + (1-H1)\ x\ [\ H2xT2 + (1-H2)\ x\ [\ H3xT3 + (1-H3)x\ [\ H4xT4 + (1-H4)xT5\ ]\ ]\ ]$$

- **sequential access**:

$$EAT_5 = H1xT1 + (1-H1)\ x\ [\ H2x(T1+T2\ ) + (1-H2)\ x\ [\ H3x(T1+T2+T3)+ (1-H3)x(*)]\ ]$$

$$EAT_5 = H1xT1 + (1-H1)\ x\ [\ H2x(T1+T2\ ) + (1-H2)\ x\ [\ H3x(T1+T2+T3)+ (1-H3)x\ [\ H4x(T1+T2+T3+T4) + (1-H4)x(T1+T2+T3+T4+T5)\ ]]\ ]$$

EAT3 a) Deduce the EAT expression for a 5-level memory hierarchy. b) Apply it reusing the times and probabilities of exercise EAT2, and considering a hit-rate in level 4 of 99% and access time of 1 ms in level 5.

Solution:

b)

H1 = 0,90; *H2* = 0,94; *H3* = 0,97; ***H4* = 0,99**

T1 = 1 ns ; *T2* = 10 ns ; *T3* = 100 ns ; *T4* = 1000 ns ; **T5 = 1ms = 1000000ns**

- **overlapped access**:

$EAT_5 = H1xT1 + (1-H1)\ x\ [\ H2xT2 + (1-H2)\ x\ [\ H3xT3 + (1-H3)x\ [\ H4xT4 + (1-H4)xT5\ ]\ ]\ ]$

= 0,9x1 + (1-0,9) x [0,94x10 + (1-0,94) x [0,97x100 + (1-0,97)x [0,99x1000 + (1-0,99)x1000000]]] = 4,4 ns

- **sequential access**:

$EAT_5 = H1xT1 + (1-H1)\ x\ [\ H2x(T1+T2\ ) + (1-H2)\ x\ [\ H3x(T1+T2+T3)+ (1-H3)x\ [\ H4x(T1+T2+T3+T4) + (1-H4)x(T1+T2+T3+T4+T5)\ ]\ ]\ ]$

= 0,9x1 + (1-0,9) x [0,94x(1+10) + (1-0,94) x [0,97x(1+10+100) + (1-0,97)x [0,99x(1+10+100+1000) + (1-0,99) x (1+10+100+1000+ 1000000)]]]

= 4,6 ns

MP1 Consider a 2M x 32 bit memory. Answer the following questions:

a) What is the storage capacity of this memory (in bytes) ?

b) How many bits have the addresses of this memory, if the memory is byte-addressable?

c) How many bits have the addresses of this memory, if the memory is word-addressable? (note: consider the word-size as 32 bits)

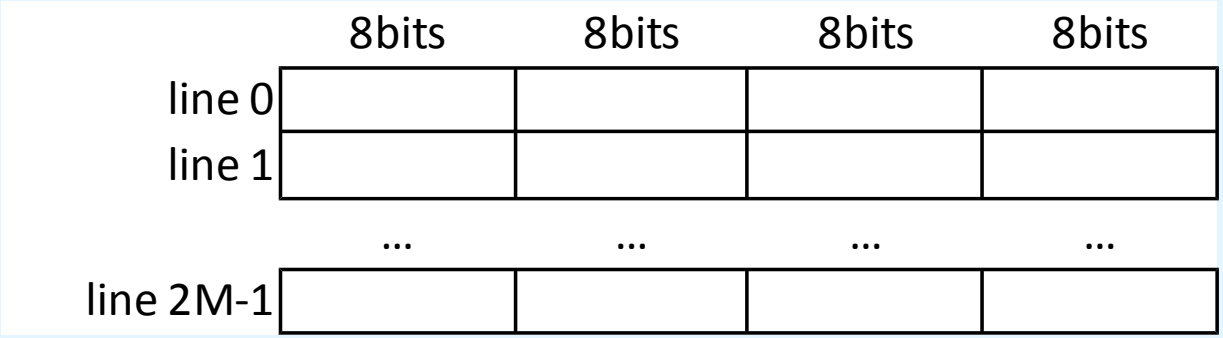
MP1 Consider a 2M x 32 bit memory. a) What is the storage capacity of this memory (in bytes) ?

Solution:

#BYTES(memory) = #LINES(memory) x #BYTES (line)
= 2M x (32 / 8)
= 2M x 4
= 8M bytes

MP1 Consider a 2M x 32 bit memory. b) How many bits have the addresses of this memory, if the memory is byte-addressable?

Solution:



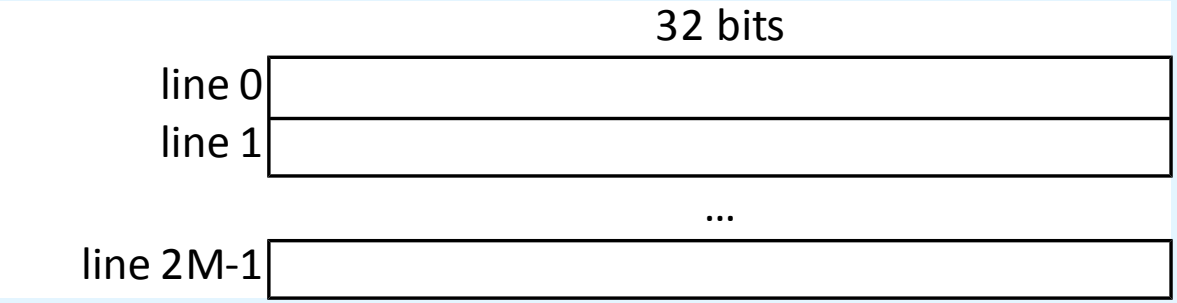
"byte-addressable memory" => CELL == 1 byte == 8bits

#CELLS(memory) = #LINES(memory) x #CELLS(line)
= 2M x 4
= 8M cells

#BITS(address) = $\log_2$(#CELLS(memory)) = $\log_2$ (8M)
= $\log_2$ (2^3 x 2^20) = $\log_2$ (2^23)
= 23 bits

MP1 Consider a 2M x 32 bit memory. c) How many bits have the addresses of this memory, if the memory is word-addressable? (note: consider the word-size as 32 bits)

Solution:



"word-addressable memory" => CELL == 32bits == 4bytes

#CELLS(memory) = #LINES(memory) x #CELLS(line)
= 2M x 1
= 2M cells

#BITS(address) = $\log_2$ (#CELLS(memory)) = $\log_2$ (2M)
= $\log_2$ (2^1 x 2^20) = $\log_2$ (2^21)
= 21 bits

MP2 Consider a 4M x 16 bit memory. Answer the following questions:

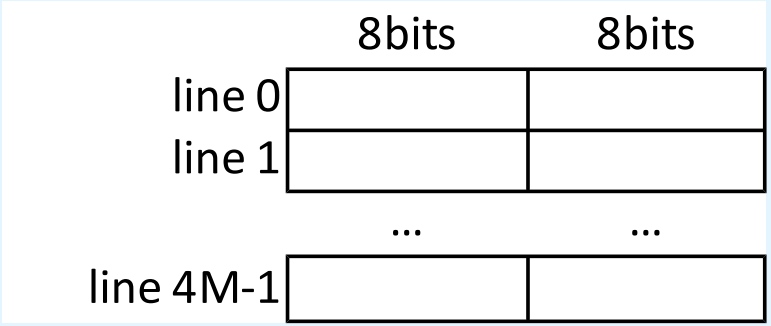
a) What is the storage capacity of this memory (in bytes) ?

b) How many bits have the addresses of this memory, if the memory is byte-addressable?

c) How many bits have the addresses of this memory, if the memory is word-addressable? (note: consider the word-size as 16 bits)

MP2 Consider a 4M x 16 bit memory. a) What is the storage capacity of this memory (in bytes) ?

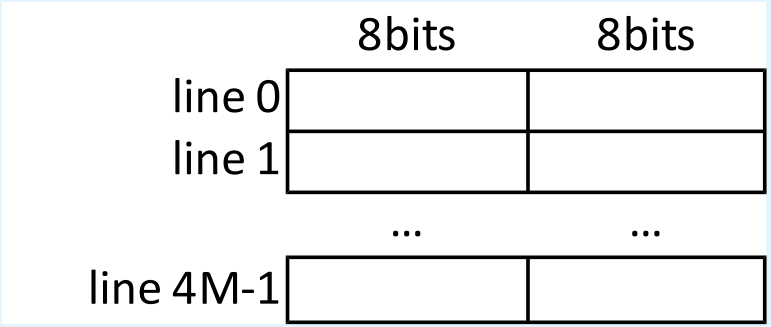
Solution:



#BYTES(memory) = #LINES(memory) x #BYTES (line)
= 4M x (16 / 8)
= 4M x 2
= 8M bytes

MP2 Consider a 4M x 16 bit memory. b) How many bits have the addresses of this memory, if the memory is byte-addressable?

Solution:



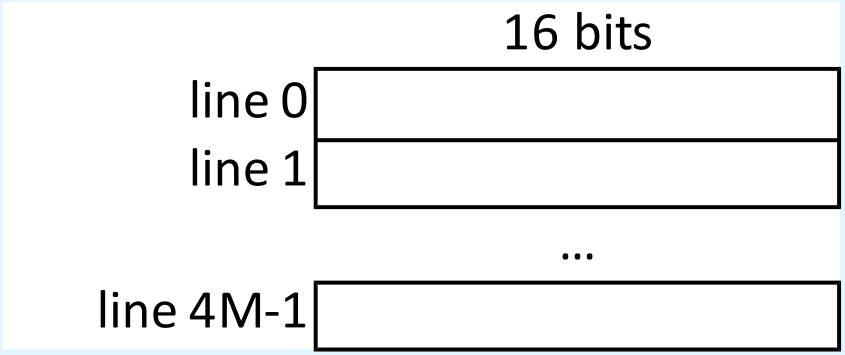
"byte-addressable memory" => CELL == 1 byte == 8bits

#CELLS(memory) = #LINES(memory) x #CELLS(line)
= 4M x 2
= 8M cells

#BITS(address) = $\log_2$(#CELLS(memory)) = $\log_2$ (8M)
= $\log_2$ (2^3 x 2^20) = $\log_2$ (2^23)
= 23 bits

MP2 Consider a 4M x 16 bit memory. c) How many bits have the addresses of this memory, if the memory is word-addressable? (note: consider the word-size as 16 bits)

Solution:



"word-addressable memory" => CELL == 16bits == 2bytes

#CELLS(memory) = #LINES(memory) x #CELLS(line)
= 4M x 1
= 4M cells

#BITS(address) = $\log_2$ (#CELLS(memory)) = $\log_2$ (4M)
= $\log_2$ (2^2 x 2^20) = $\log_2$ (2^22)
= 22 bits

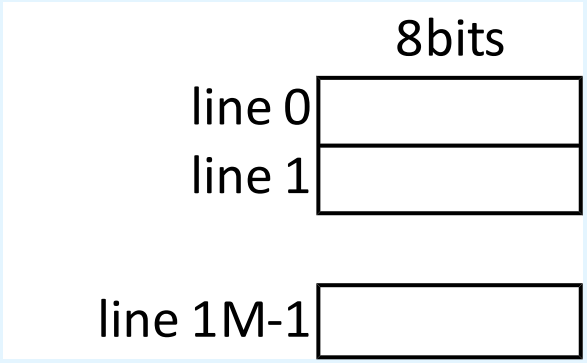
MP3 Consider a 1M x 8 bit memory. Answer the following questions:

a) How many bits have the addresses of this memory, if the memory is byte-addressable?

b) How many bits have the addresses of this memory, if the memory is word-addressable?

MP3 Consider a 1M x 8 bit memory. a) How many bits have the addresses of this memory, if the memory is byte-addressable?

Solution:



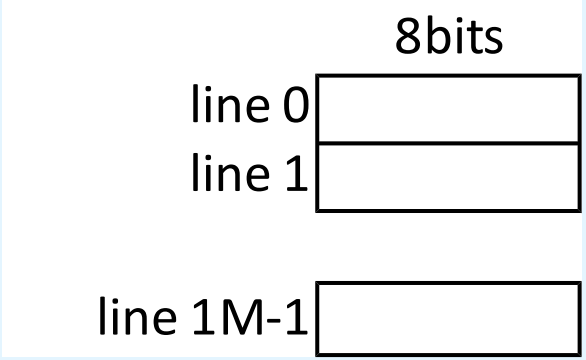
"byte-addressable memory" => CELL == 1 byte == 8bits

#CELLS(memory) = #LINES(memory) x #CELLS(line)
= 1M x 1
= 1M cells

#BITS(address) = $\log_2$(#CELLS(memory)) = $\log_2$ (1M)
= $\log_2$ (2^0 x 2^20) = $\log_2$ (2^20)
= 20 bits

MP3 Consider a 1M x 8 bit memory. b) How many bits have the addresses of this memory, if the memory is word-addressable?

Solution:



"word-addressable memory" => CELL == 8bits == 1bytes
(in this scenario, the word is restricted to the minimum of a single byte)

#CELLS(memory) = #LINES(memory) x #CELLS(line)
= 1M x 1
= 1M cells

#BITS(address) = $\log_2$ (#CELLS(memory)) = $\log_2$ (1M)
= $\log_2$ (2^0 x 2^20) = $\log_2$ (2^20)
= 20 bits

MP4 Consider a memory of 2^{20} bytes. Answer the following questions:

a) What is the lowest and highest address if this memory is byte-addressable?

b) What is the lowest and highest address if this memory is word-addressable? (note: consider the word size as 16 bits)

c) What is the lowest and highest address if this memory is word-addressable? (note: consider the word size as 32 bits)

MP4 Consider a memory of 2^{20} bytes: a) What is the lowest and highest address if this memory is byte-addressable?

Solution:

"byte addressable memory" => #BYTES(cell) = 1 byte

#CELLS(memory) = #BYTES(memory) / #BYTES(cell)
= (2^20) / 1
= 2^20 cells

address range = 0 … #CELLS(memory) - 1 = **0** … **2^20 -1**

MP4 Consider a memory of 2^{20} bytes: b) What is the lowest and highest address if this memory is word-addressable? (note: consider the word size as 16 bits)

Solution:

“16-bit word addressable memory” => #BYTES(cell) = 2 bytes

#CELLS(memory) = #BYTES(memory) / #BYTES(cell)
= (2^20) / 2
= 2^19 cells

address range = 0 … #CELLS(memory) - 1 = **0** … **2^19 -1**

MP4 Consider a memory of 2^{20} bytes: c) What is the lowest and highest address if this memory is word-addressable? (note: consider the word size as 32 bits)

Solution:

"32-bit word addressable memory" => #BYTES(cell) = 4 bytes

#CELLS(memory) = #BYTES(memory) / #BYTES(cell)
= (2^20) / 4
= 2^18 cells

address range = 0 … #CELLS(memory) - 1 = **0** … **2^18 -1**

MP5 A 2M x 16 main memory is built using 256K × 8 chips and memory is word-addressable. Provide:

a) The number of chips necessary per each memory word.

b) The total number of modules of the memory.

c) The total number of chips of the memory.

d) The number of bits of the memory addresses.

e) The number of bits used to select the module.

f) The number of bits used to define the offset (line) within a module.

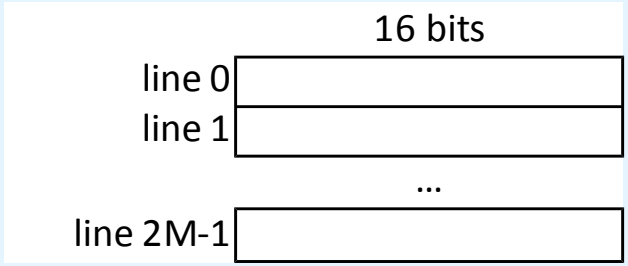
g) The module and the offset (line) within that module, for the memory address 14, under g1) High-Order and g3) Low-Order interleaving

h) The overall storage capacity (in bytes) of the memory.

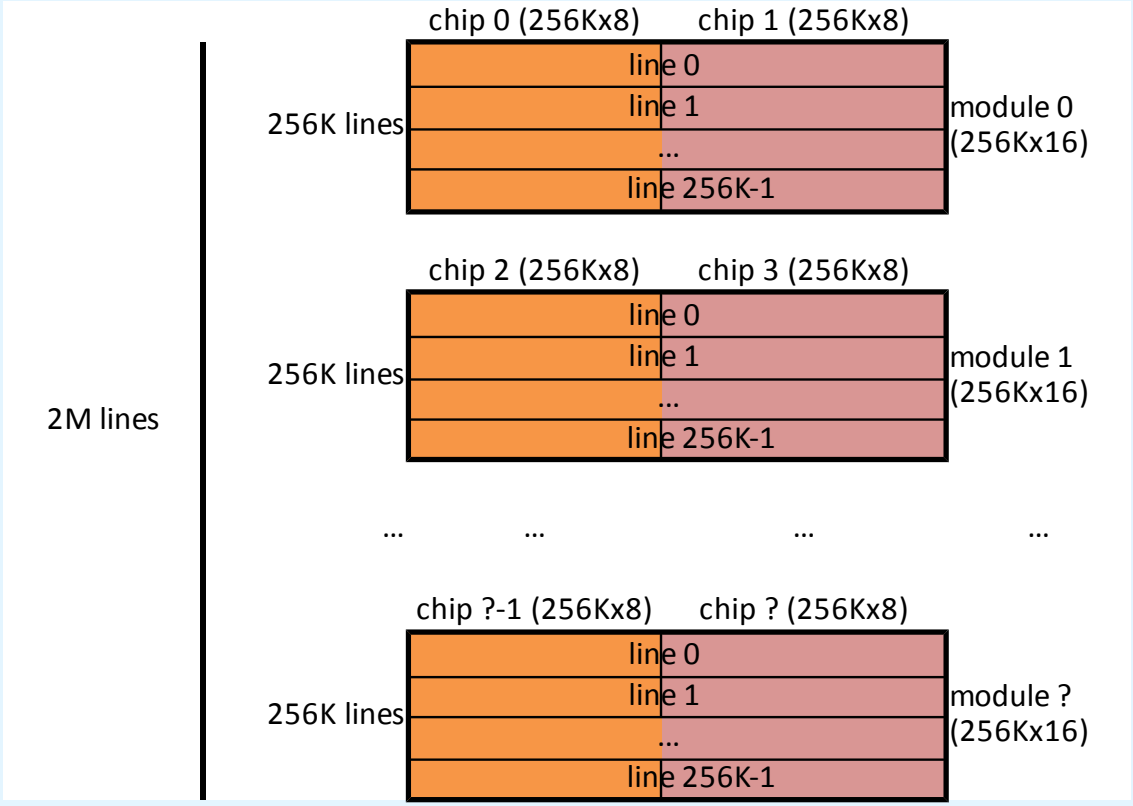
MP5 A 2M x 16 main memory is built using 256K × 8 chips and memory is word-addressable.

Based on the data given, one can build the following representations:

Abstract View of Memory



Physical View of Memory



MP5 A 2M x 16 main memory is built using 256K × 8 chips and memory is word-addressable. a) How many chips are there per memory word?

Solution:

#CHIPS(word) = #BITS(word) / #BITS(chip width)

= #BITS(memory width) / #BITS(chip width)

= 16 / 8

= **2 chips**

(therefore, **#CHIPS(module)** is also **2 chips**)

(also, a module has the same height of a chip,
that is, **#LINES(module) = #LINES(chip) = 256K**)

MP5 A 2M x 16 main memory is built using 256K × 8 chips and memory is word-addressable. b) How many RAM modules will this memory have?

Solution:

#MODULES(memory) = #LINES(memory) / **#LINES(module)**

= 2M / 256K

= 2^21 / 2^18

= 2^3

= **8 modules**

MP5 A 2M x 16 main memory is built using 256K × 8 chips and memory is word-addressable. c) How many chips will this memory have?

Solution:

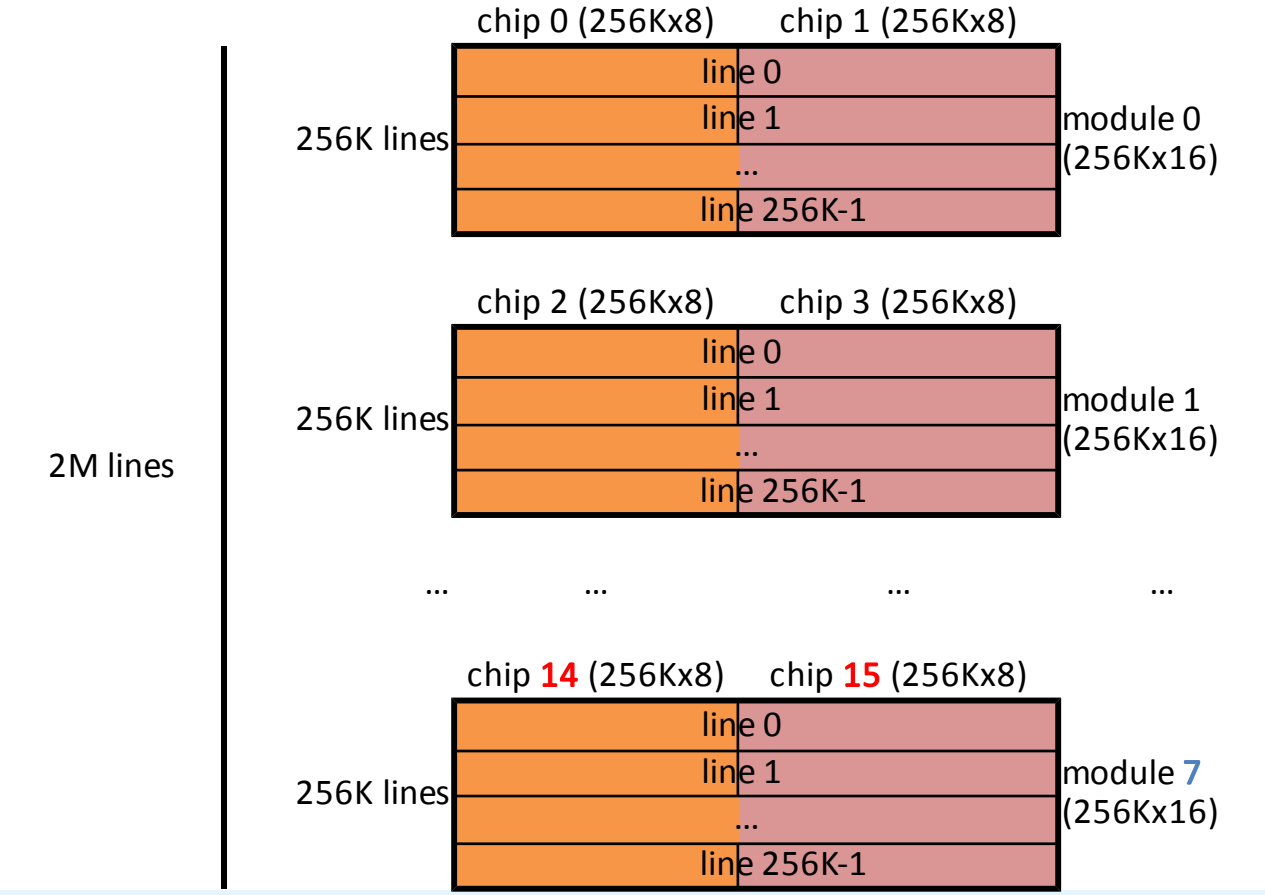
#CHIPS(memory) = **#MODULES(memory)** x **#CHIPS(module)**

= 8 x 2

= **16 chips**

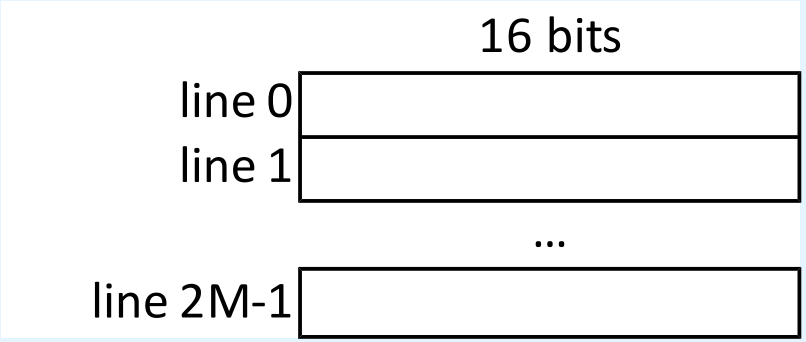
MP5 A 2M x 16 main memory is built using 256K × 8 chips and memory is word-addressable.

Accordingly with the previous calculations, this is the updated Physical View of the Memory:



MP5 A 2M x 16 main memory is built using 256K × 8 chips and memory is word-addressable. d) How many address bits are needed for all of memory?

Solution:



"word-addressable memory" => CELL == 16bits

#CELLS(memory) = #LINES(memory) x #CELLS(line)
= 2M x 1
= 2M cells

#BITS(memory address) = $\log_2$ (#CELLS(memory)) = $\log_2$ (2M)
= $\log_2$ (2^1 x 2^20) = $\log_2$ (2^21)
= 21 bits

MP5 A 2M x 16 main memory is built using 256K × 8 chips and memory is word-addressable. e) How many address bits are needed to identify a module ?

Solution:

#BITS(module address) = $\log_2$ (**#MODULES(memory)**)

= $\log_2$ (8)

= $\log_2$ (2^3)

= **3 bits**

MP5 A 2M x 16 main memory is built using 256K × 8 chips and memory is word-addressable. f) How many address bits are needed to identify a module line ?

Solution:

#BITS(module line address) = $\log_2$ (**#LINES(module)**)
= $\log_2$ (256K)
= $\log_2$ (2^8 x 2^10)
= $\log_2$ (2^18)
= **18 bits**

[or **#BITS(module line address)** = **#BITS(memory address)** - **#BITS(module address)**
= 21 – 3 = **18 bits**]

MP5 A 2M x 16 main memory is built using 256K × 8 chips and memory is word-addressable. g) Where (module and line) would address **14** be located under g1) High-Order Interleaving, g2) Low-order Interleaving ?

Solution:

14_{10}= 8+4+2 = 1110_2 = **00000000000000001110_2** (with 21 bits)

g1) with High-Order-Interleaving

	module	**module line**
14 =	**0 0 0**	**00000000000001110**

thus, **module = 0** and **module line = 14**

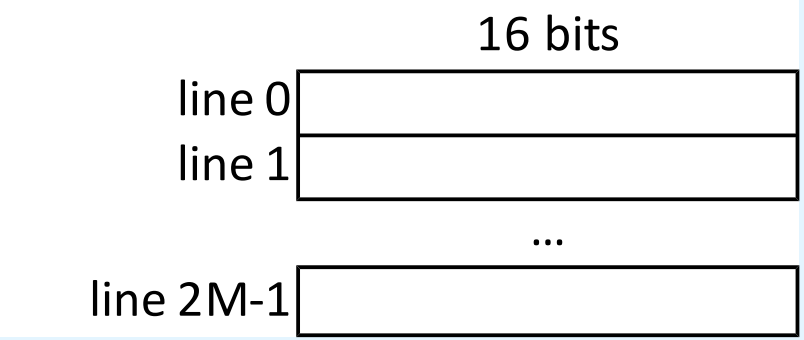
g2) with Low-Order-Interleaving

	module line	**module**
14 =	**0000000000000001**	**1 1 0**

thus, **module = 6** and **module line = 1**

MP5 A 2M x 16 main memory is built using 256K × 8 chips and memory is word-addressable. h) What is the storage capacity of this memory (in bytes) ?

Solution:



#BYTES(memory) = #LINES(memory) x #BYTES (line)
= 2M x (16 / 8)
= 2M x 2
= 4M bytes

MP6 A 8M x 32 main memory is built using 512K × 16 chips and memory is word-addressable. Provide:

a) The number of chips necessary per each memory word.

b) The total number of modules of the memory.

c) The total number of chips of the memory.

d) The number of bits of the memory addresses.

e) The number of bits used to select the module.

f) The number of bits used to define the offset (line) within a module.

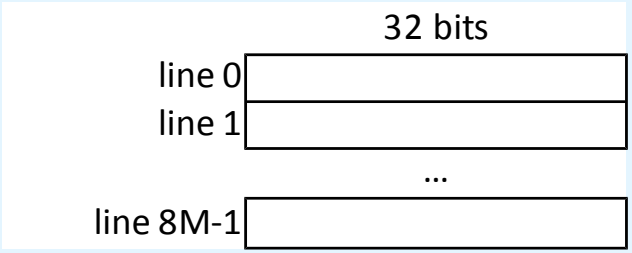
g) The module and the offset (line) within that module, for the memory address 41, under g1) High-Order and g3) Low-Order interleaving

h) The overall storage capacity (in bytes) of the memory.

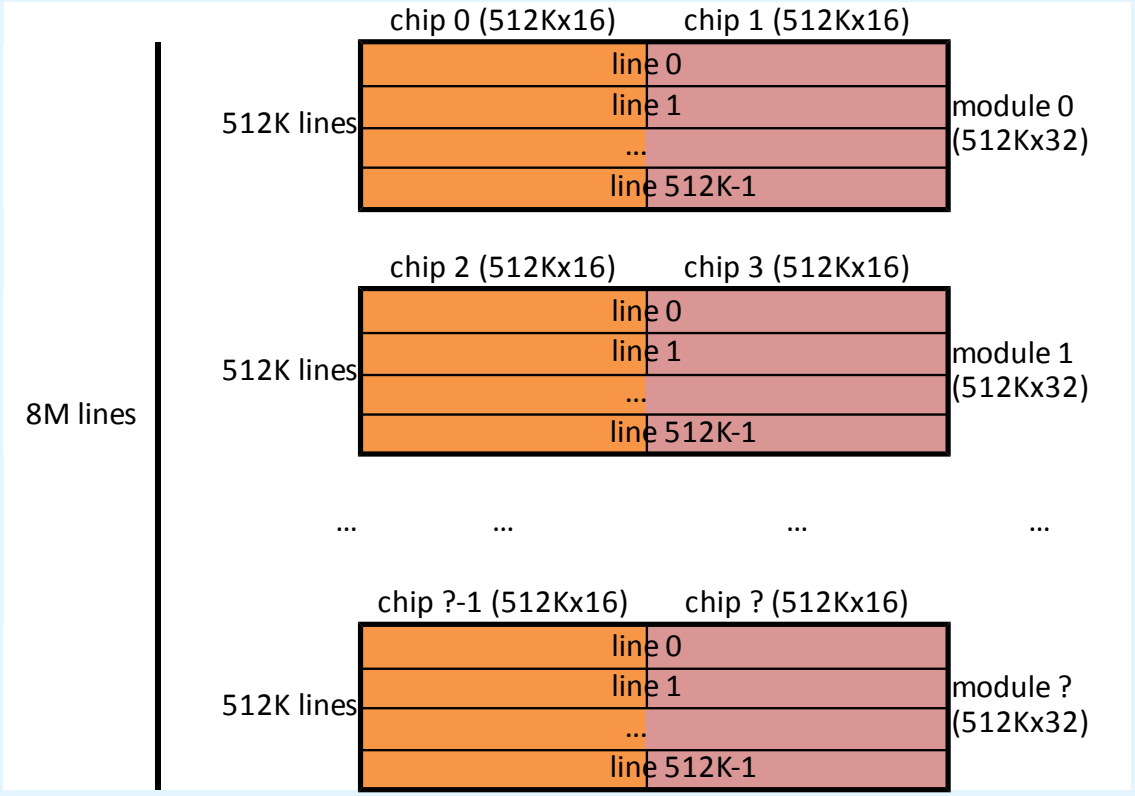
MP6 A 8M x 32 main memory is built using 512K × 16 chips and memory is word-addressable.

Based on the data given, one can build the following representations:

Abstract View of Memory



Physical View of Memory



MP6 A 8M x 32 main memory is built using 512K × 16 chips and memory is word-addressable.
a) How many chips are there per memory word?

Solution:

#CHIPS(word) = #BITS(word) / #BITS(chip width)

= #BITS(memory width) / #BITS(chip width)

= 32 / 16

= **2 chips**

(therefore, **#CHIPS(module)** is also **2 chips**)

(also, a module has the same height of a chip,
that is, **#LINES(module) = #LINES(chip) = 512K**)

MP6 A 8M x 32 main memory is built using 512K × 16 chips and memory is word-addressable.
b) How many RAM modules will this memory have?

Solution:

#MODULES(memory) = #LINES(memory) / **#LINES(module)**

= 8M / 512K

= 2^23 / 2^19

= 2^4

= **16 modules**

MP6 A 8M x 32 main memory is built using 512K × 16 chips and memory is word-addressable.
c) How many chips will this memory have?

Solution:

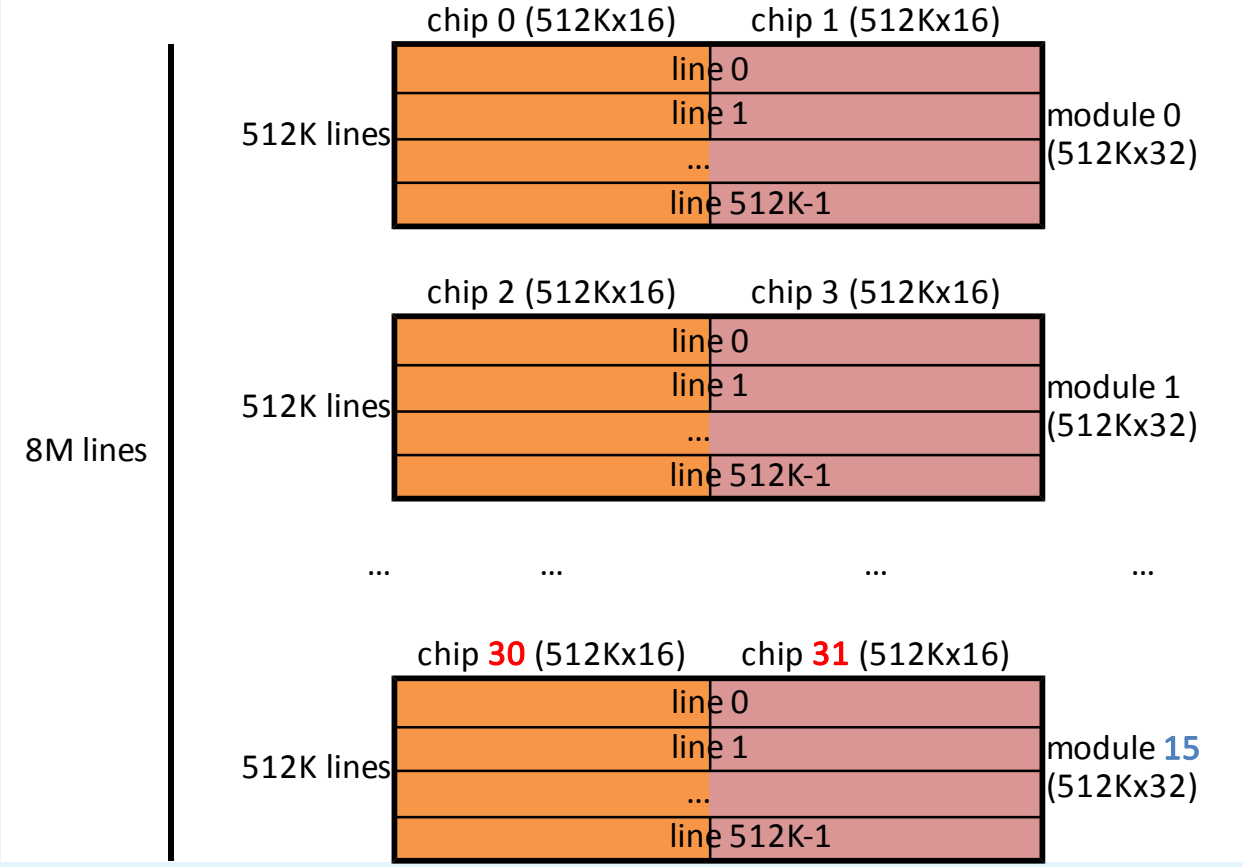
#CHIPS(memory) = **#MODULES(memory)** x **#CHIPS(module)**

= 16 x 2

= **32 chips**

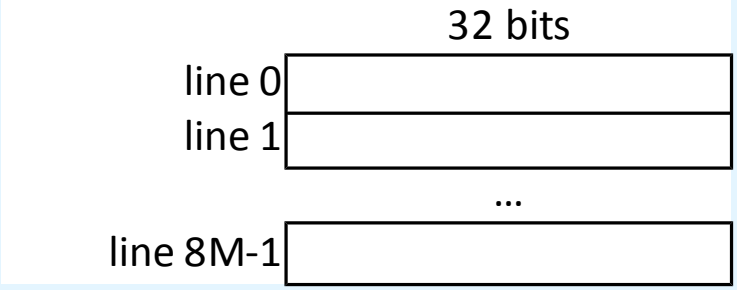
MP6 A 8M x 32 main memory is built using 512K × 16 chips and memory is word-addressable.

Accordingly with the previous calculations, this is the updated Physical View of the Memory:



MP6 A 8M x 32 main memory is built using 512K × 16 chips and memory is word-addressable. d) How many address bits are needed for all of memory?

Solution:



"word-addressable memory" => CELL == 32bits

#CELLS(memory) = #LINES(memory) x #CELLS(line)
= 8M x 1
= 8M cells

#BITS(memory address) = $\log_2$ (#CELLS(memory)) = $\log_2$ (2M)
= $\log_2$ (2^3 x 2^20) = $\log_2$ (2^23)
= 23 bits

MP6 A 8M x 32 main memory is built using 512K × 16 chips and memory is word-addressable. e) How many address bits are needed to identify a module ?

Solution:

#BITS(module address) = $\log_2$ (**#MODULES(memory)**)

= $\log_2$ (16)

= $\log_2$ (2^4)

= **4 bits**

MP6 A 8M x 32 main memory is built using 512K × 16 chips and memory is word-addressable. f) How many address bits are needed to identify a module line ?

Solution:

#BITS(module line address) = $\log_2$ (**#LINES(module)**)
= $\log_2$ (512K)
= $\log_2$ (2^9 x 2^10)
= $\log_2$ (2^19)
= **19 bits**

[or **#BITS(module line address)** = **#BITS(memory address)** - **#BITS(module address)**
= 23 – 4 = **19 bits**]

MP6 A 8M x 32 main memory is built using 512K × 16 chips and memory is word-addressable. g) Where (module and line) would address **41** be located under g1) High-Order Interleaving, g2) Low-order Interleaving ?

Solution:

$\mathbf{41_{10}}$= 32+8+1 = 101001_2 = $\mathbf{00000000000000000101001_2}$ (with 23 bits)

g1) with High-Order-Interleaving

	module	module line
41 =	**0 0 0 0**	**00000000000000101001**

thus, **module = 0** and **module line = 41**

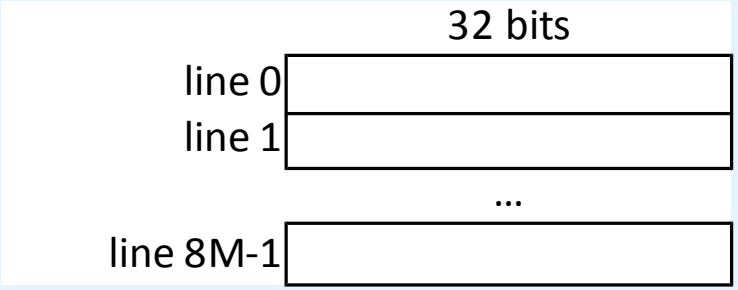
g2) with Low-Order-Interleaving

	module line	module
41 =	**00000000000000010**	**1 0 0 1**

thus, **module = 9** and **module line = 2**

MP6 A 8M x 32 main memory is built using 512K × 16 chips and memory is word-addressable. h) What is the storage capacity of this memory (in bytes) ?

Solution:



#BYTES(memory) = #LINES(memory) x #BYTES (line)
= 8M x (32 / 8)
= 8M x 4
= 32M bytes

MP7 A memory of 32G x 64 bits is built with 1G x 32 chips and is word-addressable (64 bits word size). Answer the following questions.

a) Provide the number of chips necessary per each memory word:
{**?**} chips

b) Provide the total number of modules of the memory:
{**?**} modules

c) Provide the total number of chips of the memory:
{**?**} chips

d) Provide the number of bits of the memory addresses:
{**?**} bits

e) Provide the number of bits used to select the module:
{**?**} bits

f) Provide the number of bits used to define the offset (line) within a module:
{**?**} bits

MP7 A memory of 32G x 64 bits is built with 1G x 32 chips and is word-addressable (64 bits word size). Answer the following questions.

Solution:

a) Provide the number of chips necessary per each memory word:
{**2**} chips

b) Provide the total number of modules of the memory:
{**32**} modules

c) Provide the total number of chips of the memory:
{**64**} chips

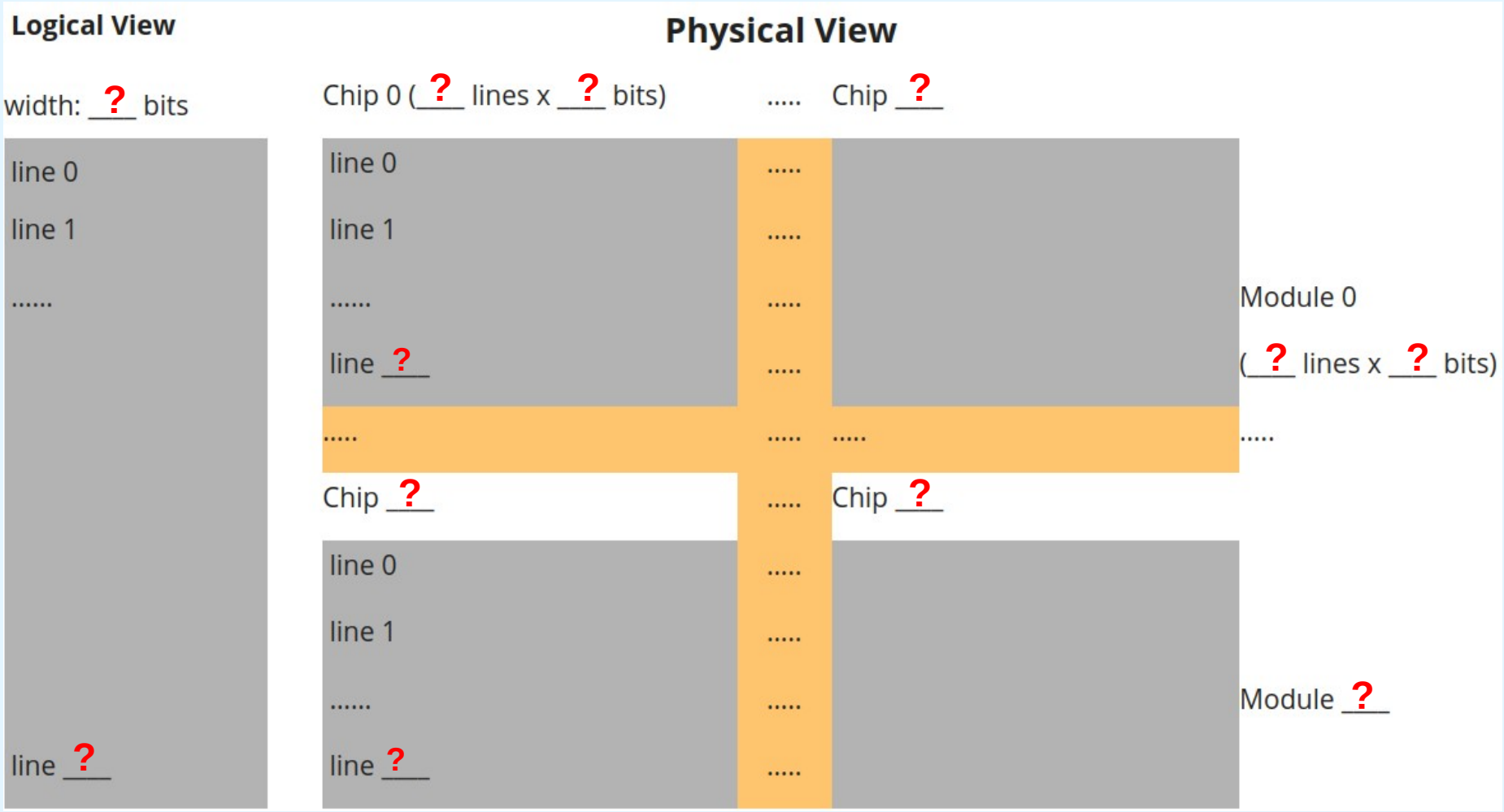
d) Provide the number of bits of the memory addresses:
{**35**} bits

e) Provide the number of bits used to select the module:
{**5**} bits

f) Provide the number of bits used to define the offset (line) within a module:
{**30**} bits

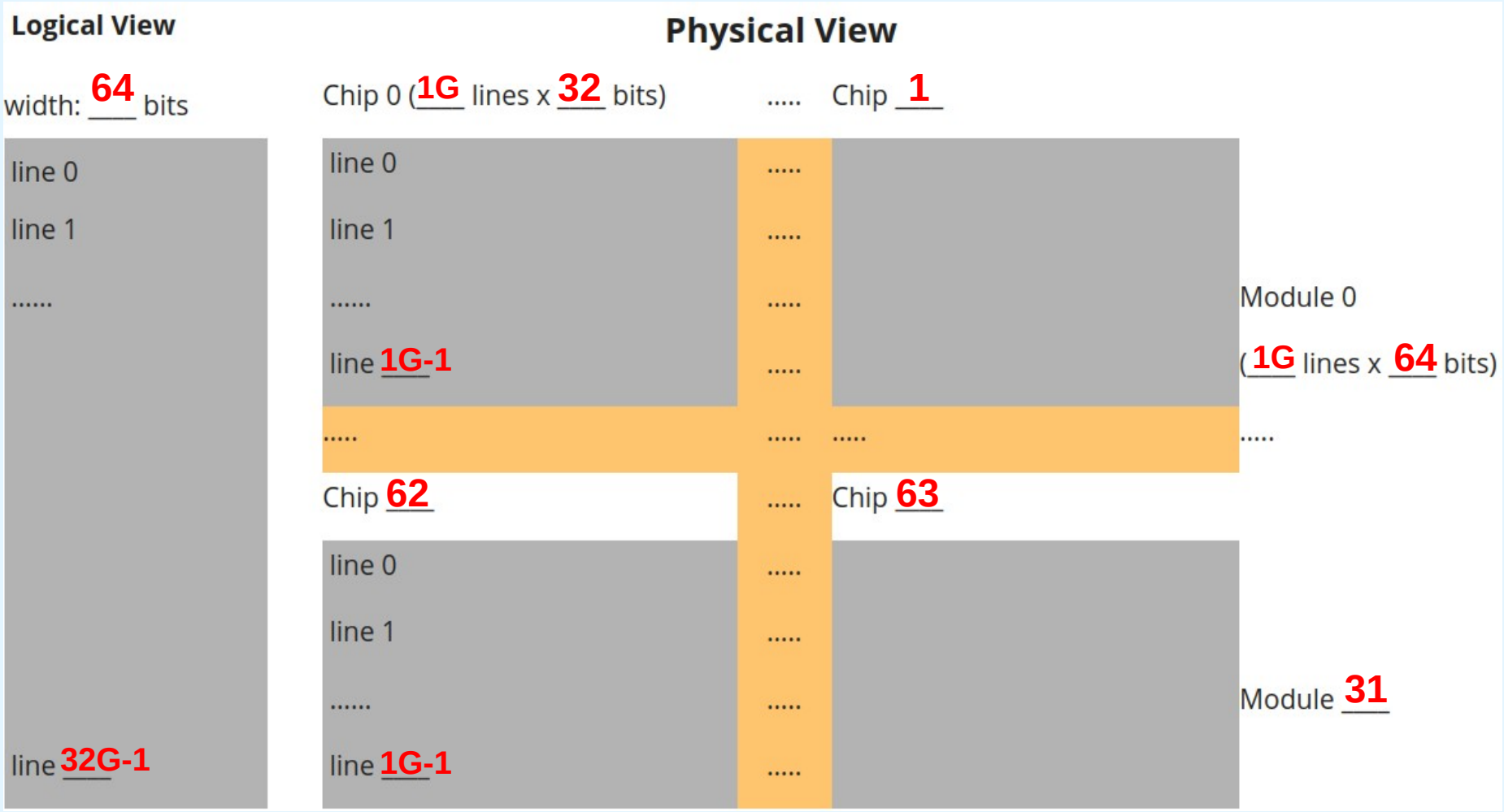
MP7 A memory of 32G x 64 bits is built with 1G x 32 chips and is word-addressable (64 bits word size). Answer the following questions.

g) Complete the representation bellow of the logical & physical view of memory:



MP7 A memory of 32G x 64 bits is built with 1G x 32 chips and is word-addressable (64 bits word size). Answer the following questions.

g) Complete the representation bellow of the logical & physical view of memory:
Solution:



MP7 A memory of 32G x 64 bits is built with 1G x 32 chips and is word-addressable (64 bits word size). Answer the following questions.

h) Find and present the module and the offset (line) within that module, for the memory address 3333, considering the following alternatives for the mapping of a memory address into its physical components:

h1) High-Order Interleaving:

module: {**?**}
offset: {**?**}

h2) Low-Order Interleaving:

module: {**?**}
offset: {**?**}

MP7 A memory of 32G x 64 bits is built with 1G x 32 chips and is word-addressable (64 bits word size). Answer the following questions.

h) Find and present the module and the offset (line) within that module, for the memory address 3333, considering the following alternatives for the mapping of a memory address into its physical components:

Solution:

h1) High-Order Interleaving:

module: {**0**}
offset: {**3333**}

h2) Low-Order Interleaving:

module: {**5**}
offset: {**104**}

MP7 A memory of 32G x 64 bits is built with 1G x 32 chips and is word-addressable (64 bits word size). Answer the following questions.

i) Provide the overall storage capacity (in bytes) of the memory.

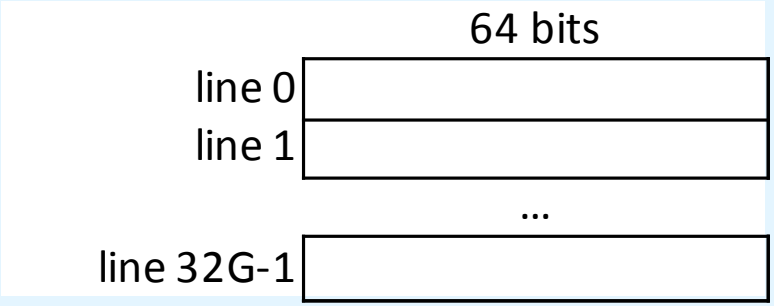
{**?**} bytes

MP7 A memory of 32G x 64 bits is built with 1G x 32 chips and is word-addressable (64 bits word size). Answer the following questions.

i) Provide the overall storage capacity (in bytes) of the memory.

Solution:

{**256G**} bytes



#BYTES(memory) = #LINES(memory) x #BYTES (line)
= 32G x (64 / 8)
= 32G x 8
= 256G bytes

MP8 A memory of 16G x 32 bits is built with 256M x 8 chips and is word-addressable (32 bits word size). Answer the following questions.

a) Provide the number of chips necessary per each memory word:
{**?**} chips

b) Provide the total number of modules of the memory:
{**?**} modules

c) Provide the total number of chips of the memory:
{**?**} chips

d) Provide the number of bits of the memory addresses:
{**?**} bits

e) Provide the number of bits used to select the module:
{**?**} bits

f) Provide the number of bits used to define the offset (line) within a module:
{**?**} bits

MP8 A memory of 16G x 32 bits is built with 256M x 8 chips and is word-addressable (32 bits word size). Answer the following questions.

Solution:

a) Provide the number of chips necessary per each memory word:
{**4**} chips

b) Provide the total number of modules of the memory:
{**64**} modules

c) Provide the total number of chips of the memory:
{**256**} chips

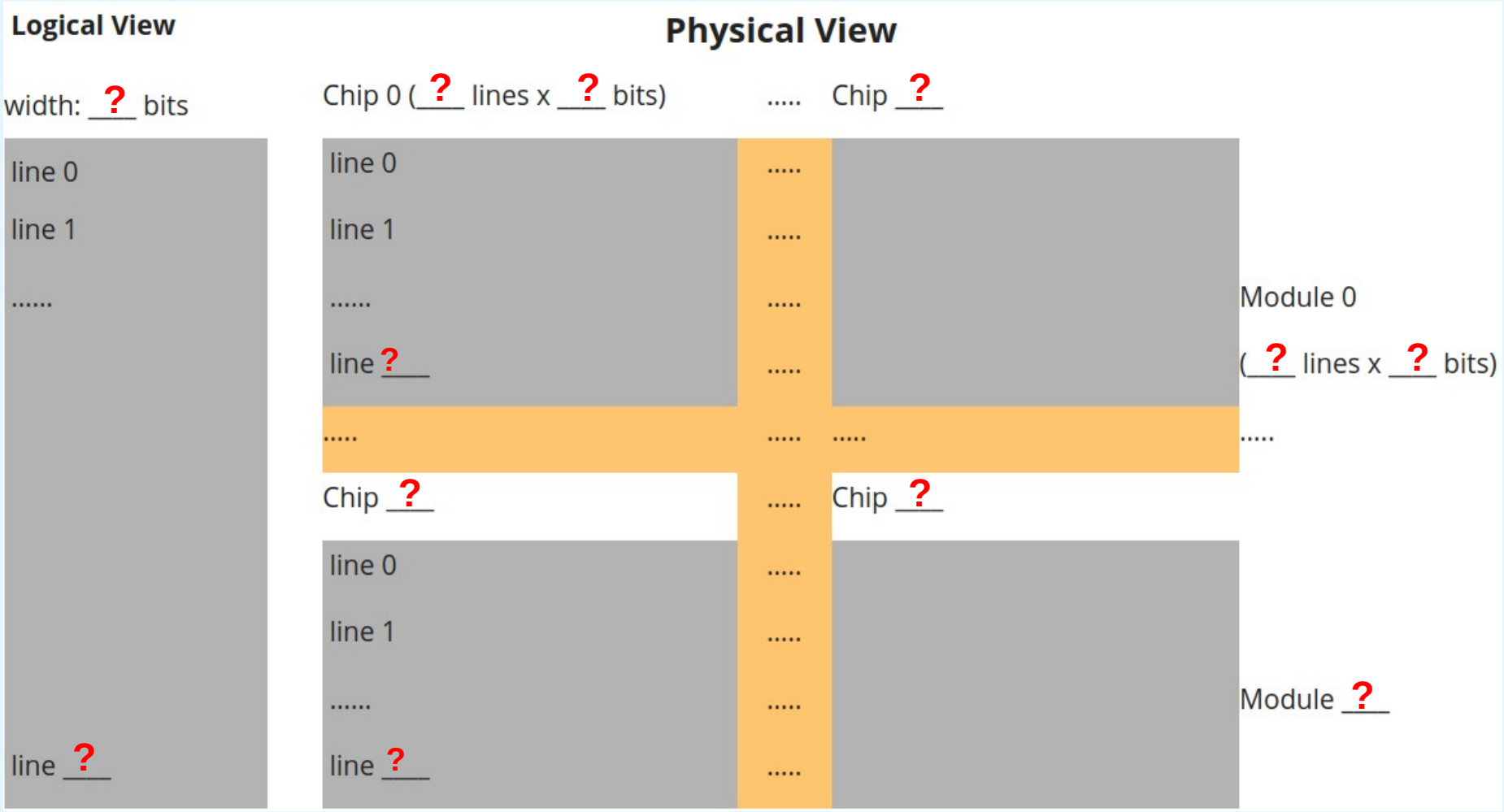
d) Provide the number of bits of the memory addresses:
{**34**} bits

e) Provide the number of bits used to select the module:
{**6**} bits

f) Provide the number of bits used to define the offset (line) within a module:
{**28**} bits

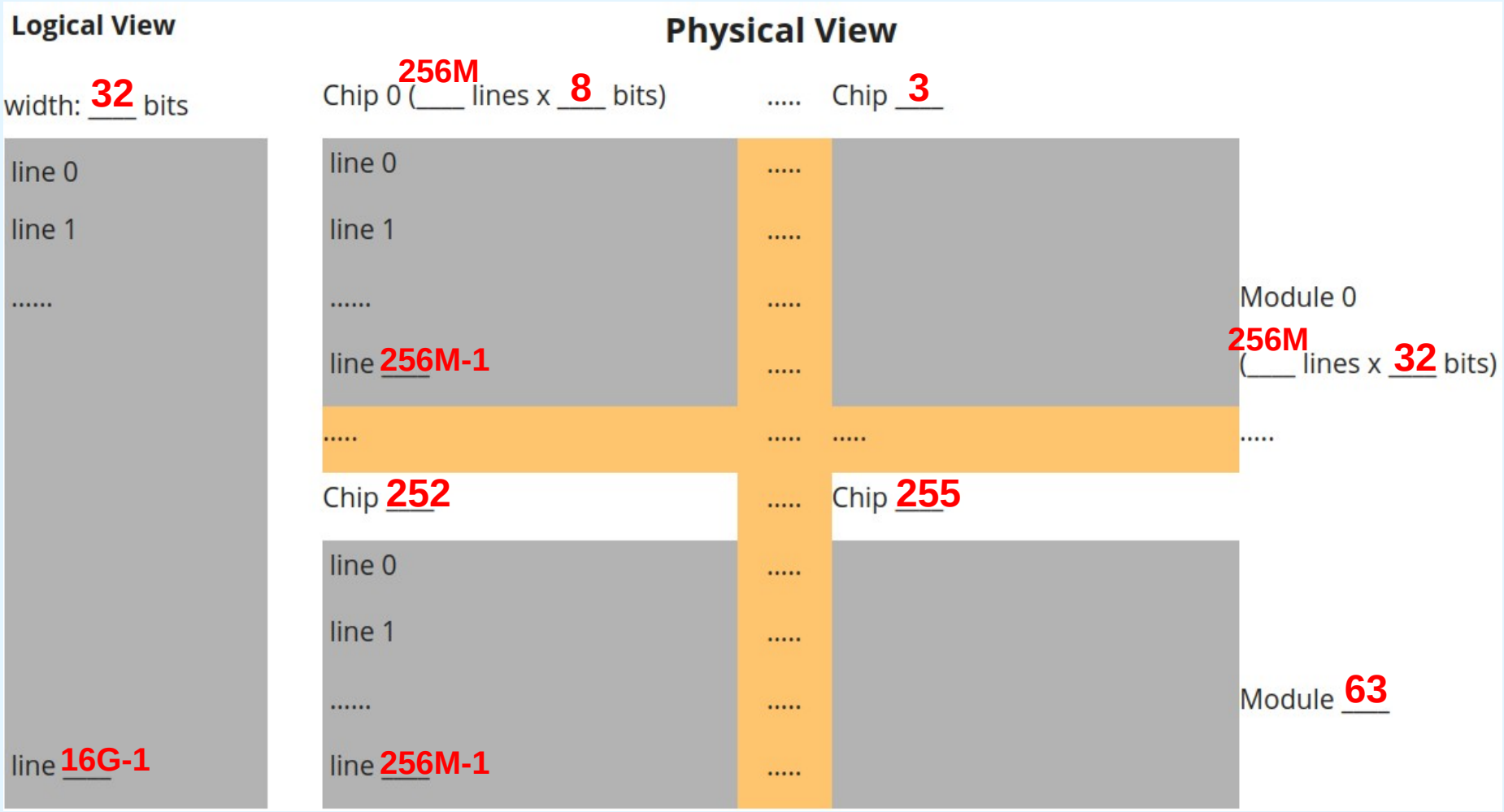
MP8 A memory of 16G x 32 bits is built with 256M x 8 chips and is word-addressable (32 bits word size). Answer the following questions.

g) Complete the representation bellow of the logical & physical view of memory:



MP8 A memory of 16G x 32 bits is built with 256M x 8 chips and is word-addressable (32 bits word size). Answer the following questions.

g) Complete the representation bellow of the logical & physical view of memory:
Solution:



MP8 A memory of 16G x 32 bits is built with 256M x 8 chips and is word-addressable (32 bits word size). Answer the following questions.

h) Find and present the module and the offset (line) within that module, for the memory address 8888888, considering the following alternatives for the mapping of a memory address into its physical components:

h1) High-Order Interleaving:

module: {**?**}
offset: {**?**}

h2) Low-Order Interleaving:

module: {**?**}
offset: {**?**}

MP8 A memory of 16G x 32 bits is built with 256M x 8 chips and is word-addressable (32 bits word size). Answer the following questions.

h) Find and present the module and the offset (line) within that module, for the memory address 8888888, considering the following alternatives for the mapping of a memory address into its physical components:

Solution:

h1) High-Order Interleaving:

module: {**0**}
offset: {**8888888**}

h2) Low-Order Interleaving:

module: {**56**}
offset: {**138888**}

MP8 A memory of 16G x 32 bits is built with 256M x 8 chips and is word-addressable (32 bits word size). Answer the following questions.

i) Provide the overall storage capacity (in bytes) of the memory.

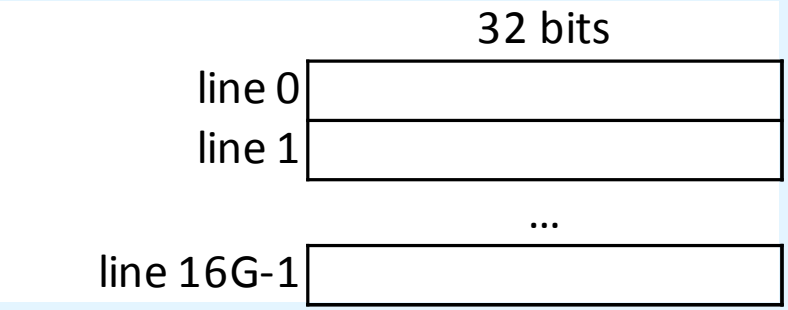
{**?**} bytes

MP8 A memory of 16G x 32 bits is built with 256M x 8 chips and is word-addressable (32 bits word size). Answer the following questions.

i) Provide the overall storage capacity (in bytes) of the memory.

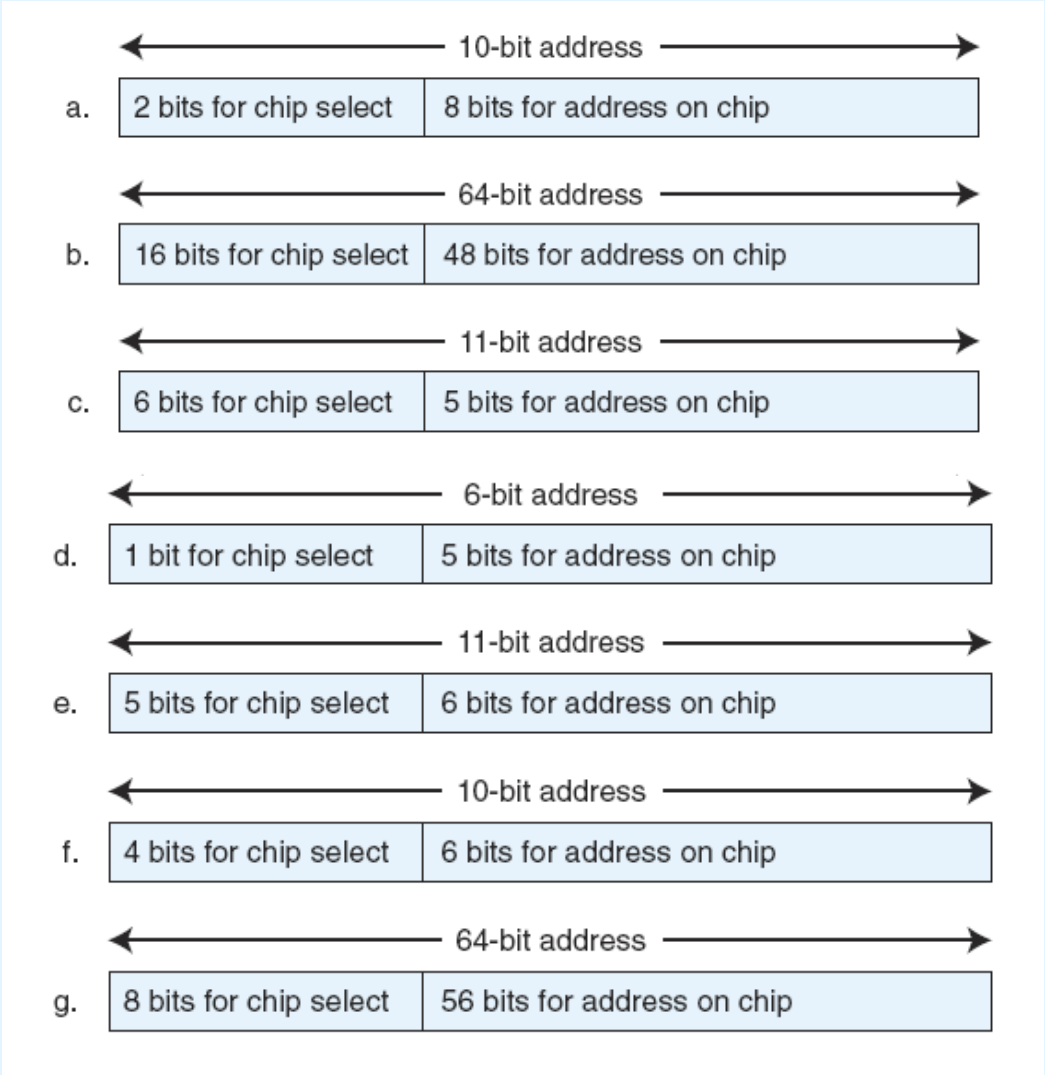
Solution:

{**64G**} bytes



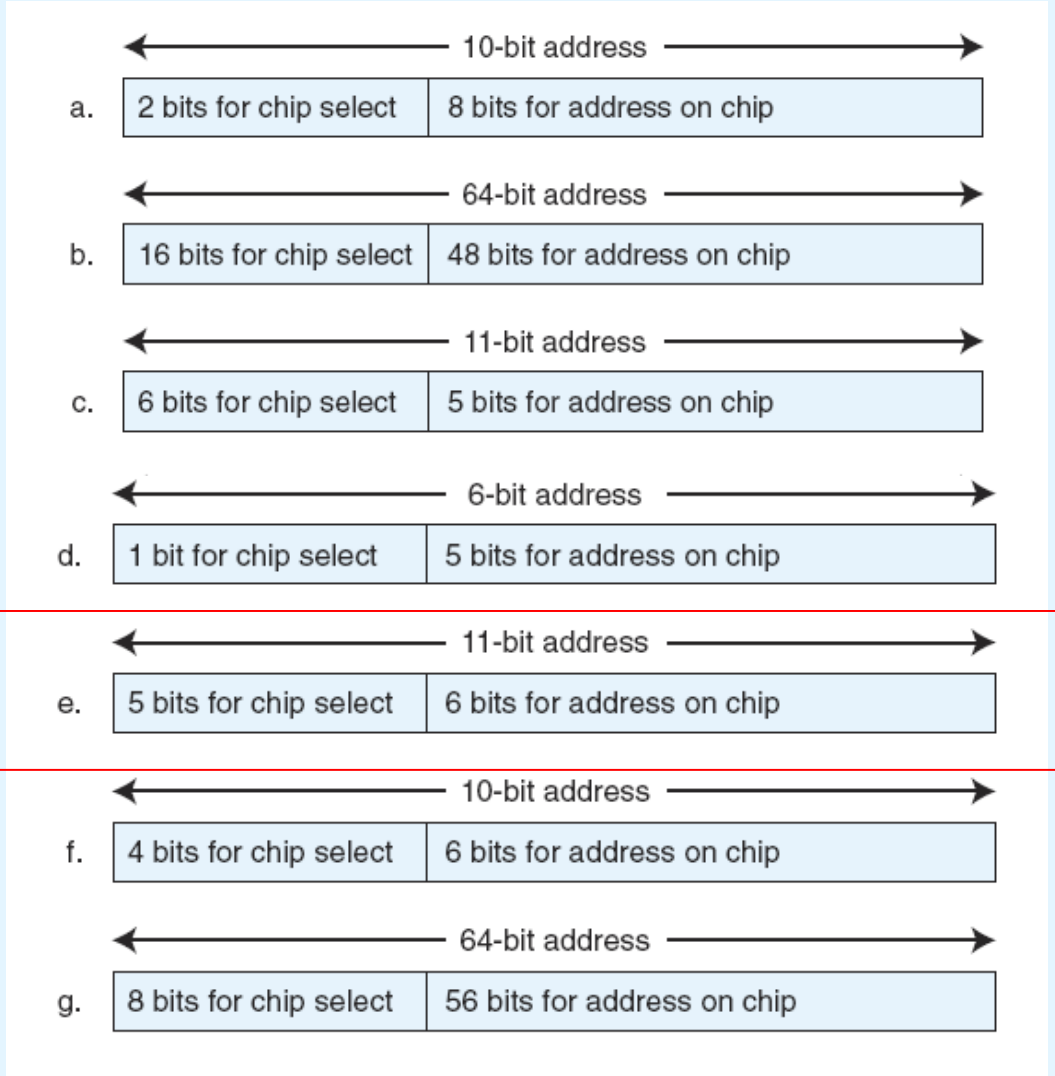
#BYTES(memory) = #LINES(memory) x #BYTES (line)
= 16G x (32 / 8)
= 16G x 4
= 64G bytes

MP9 Given a memory of 2048 bytes consisting of several 64 Byte x 8 chips, and assuming byte-addressable memory, which of the following seven alternatives uses the address bits correctly ?



MP9 Given a memory of 2048 bytes consisting of several 64 Byte x 8 chips, and assuming byte-addressable memory, which of the following seven alternatives uses the address bits correctly ?

Solution:



CACHE1 A computer has a main memory of 2^20 words and a direct mapped cache of 32 blocks, where each cache block contains 16 words.

a) **How many blocks of memory are there ?**

b) **What is the format of a memory address as seen by the cache ?**

c) **To which cache block will the memory address $0DB63_{16}$ map ?**

d) **To which memory block and word will the memory address $0DB63_{16}$ map ?**

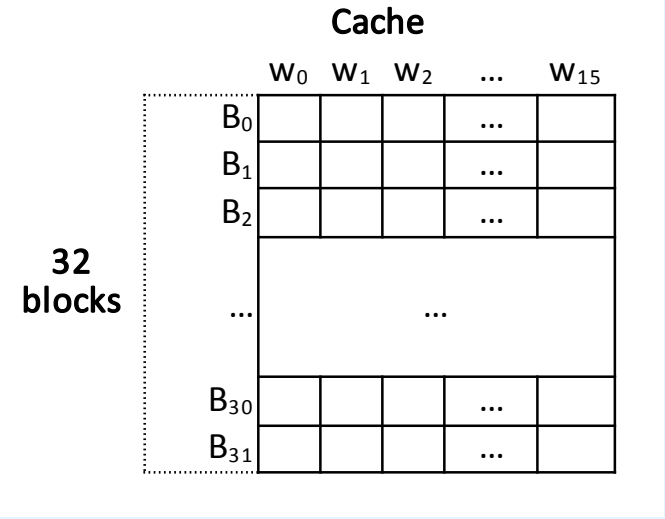
Note: in this and in the following exercises on caches, “memory” refers to the “main memory”, also sometimes referred to as “RAM”.

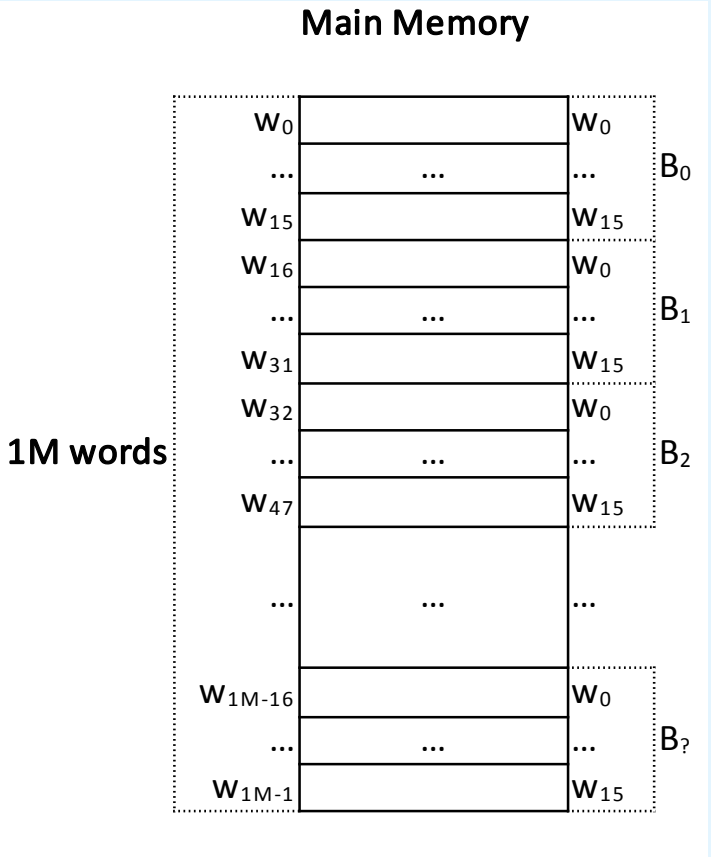
CACHE1 A computer has a main memory of 2^20 words and a direct mapped cache of 32 blocks, where each cache block contains 16 words.

Based on the data given, one can build the following representations:

Problem Data:

#WORDS(memory) = 1M = 2^20 words
#BLOCKS(cache) = 32 = 2^5 blocks
#WORDS(cache block) = 16 = 2^4 words





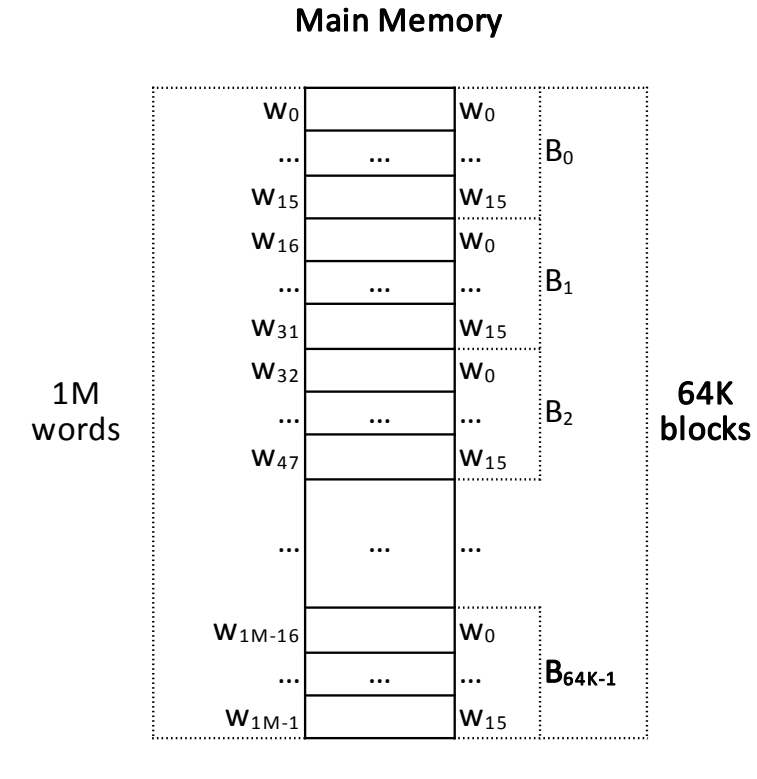
CACHE1 A computer has a main memory of 2^20 words and a direct mapped cache of 32 blocks, where each cache block contains 16 words.

a) How many blocks of memory are there?

Solution:

#WORDS(memory block)
= #WORDS(cache block)
= 16 words

#BLOCKS(memory)
= #WORDS(memory) / #WORDS(memory block)
= 2^20 / 2^4 = 2^16
= 65536 = **64K blocks**



CACHE1 A computer has a main memory of 2^20 words and a direct mapped cache of 32 blocks, where each cache block contains 16 words.

b) What is the format of a memory address as seen by the cache ?

Solution:

- cache view: **memory address=<tag, block, word>**
- **#BITS(memory address)** = log2(#WORDS(memory)) = log2(2^20) = 20 bits
- **#BITS(word)** = log2(#WORDS(cache block)) = log2(16) = **4 bits**
- **#BITS(block)** = log2(#BLOCKS(cache)) = log2(32) = **5 bits**
- **#BITS(tag)** = #BITS(memory address) – (#BITS(word) + #BITS(block))
  = 20 – (4 + 5)
  = **11 bits**

CACHE1 A computer has a main memory of 2^20 words and a direct mapped cache of 32 blocks, where each cache block contains 16 words.

b) What is the format of a memory address as seen by the cache ?

Solution (cont.):

20 bits

tag	block	word
11 bits	5 bits	4 bits
2048 groups, of 32 blocks each	32 blocks, of 16 words each	16 words

Additional notes:

- the mapping between memory blocks and cache blocks repeats every **32** memory blocks (**block = block memory % 32**) and such happens **2048** times
- thus, memory blocks 0,1,…,31 are mapped to the cache blocks 0,1,…,31; memory blocks 32,33,…,63 are mapped to cache blocks 0,1,…,31; and so on ...
- the **tag** field of a cache block states which memory block was copied to that cache block, from **2048** possible candidates

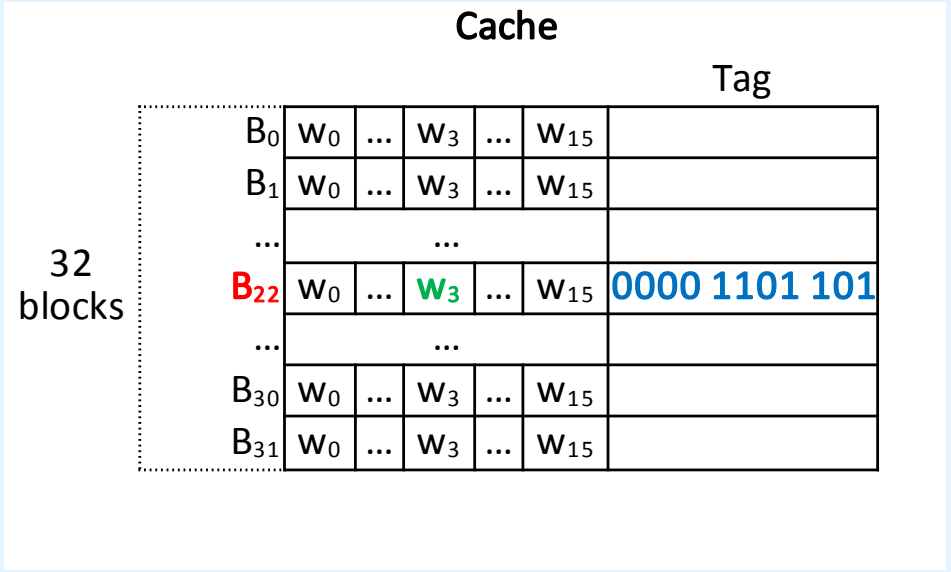
CACHE1 A computer has a main memory of 2^20 words and a direct mapped cache of 32 blocks, where each cache block contains 16 words.

c) To which cache block will the memory address $0DB63_{16}$ map?

Solution: $0DB63_{16} = 0000\ 1101\ 1011\ 0110\ 0011_{2}$

20 bits

tag	block	word
11 bits	5 bits	4 bits
0000 1101 101	1 0110	0011
(tag 109)	(block 22)	(word 3)



CACHE1 A computer has a main memory of 2^20 words and a direct mapped cache of 32 blocks, where each cache block contains 16 words.

d) To which memory block and word will the memory address $0DB63_{16}$ map?

Solution:

- cache view:

$$0DB63_{16} = 0000\ 1101\ 1011\ 0110\ 0011_{2} = 56163_{10}$$

- memory view:

memory address=<memory block, word>

- the **memory block** is found by joining the **tag** and **block** bits of the address from the cache view, that is:

memory block = $000\ 1101\ 1011\ 0110_{2} = 3510_{10}$

- the **word** component is given by the remaining bits:

word = $0011_{2} = 3$

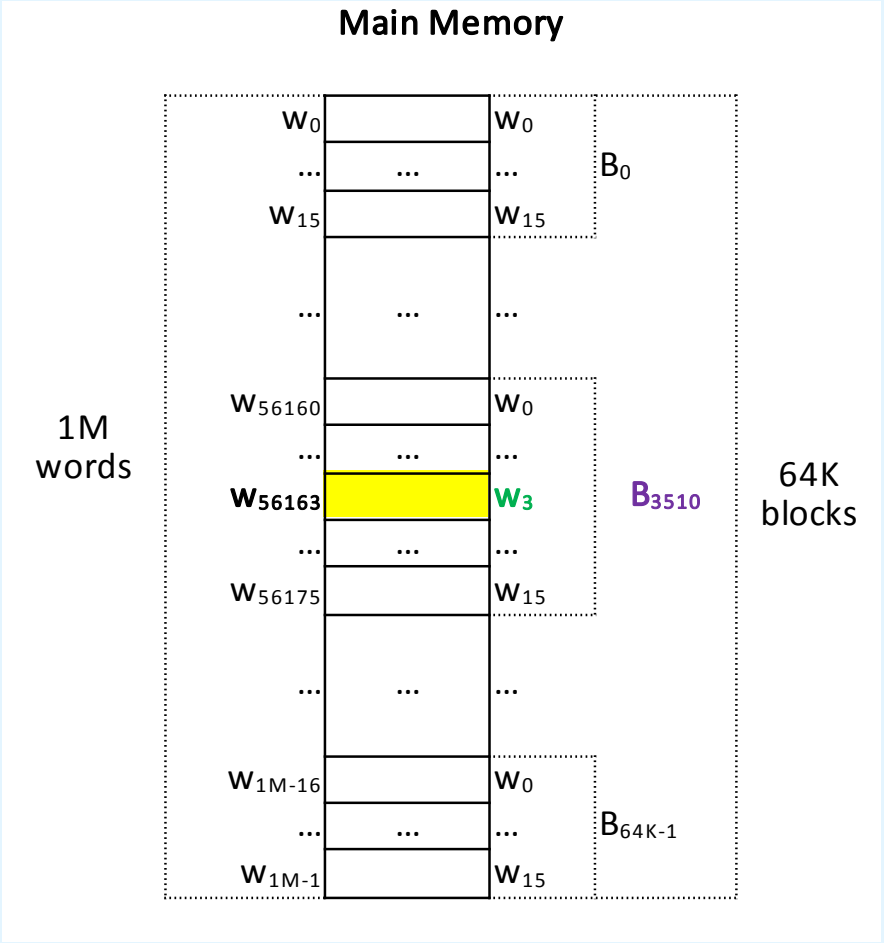
CACHE1 A computer has a main memory of 2^20 words and a direct mapped cache of 32 blocks, where each cache block contains 16 words.

d) To which memory block and word will the memory address $0DB63_{16}$ map?

Solution (cont.):

Notice:

- 56160 = 3510 x 16
- 56163 = 3510 x 16 + 3



CACHE2 A computer has a main memory of 4G words and a direct mapped cache of 1K blocks, where each cache block contains 32 words.

a) **How many blocks of memory are there?**

b) **What is the format of a memory address as seen by the cache ?**

c) **To which cache block the memory address $000063FA_{16}$ maps ?**

d) **To which memory block, and block word, the memory reference $000063FA_{16}$ maps ?**

Homework

CACHE3/EAT4 A computer has a main memory of 4K blocks (of 8 words per block) and a direct-mapped cache of 8 blocks.

a) **What is the format of a memory address as seen by the cache ?**

b) **Compute the hit ratio for a program that loops 2 times from locations 0 to 67 in memory. The cache is initially empty.**

c) **Assume access time for the cache is 22ns and the time required to fill a cache slot from main memory is 300ns. Compute the effective access time in the b) conditions.**

CACHE3/EAT4 A computer has a main memory of 4K blocks (of 8 words per block) and a direct-mapped cache of 8 blocks.

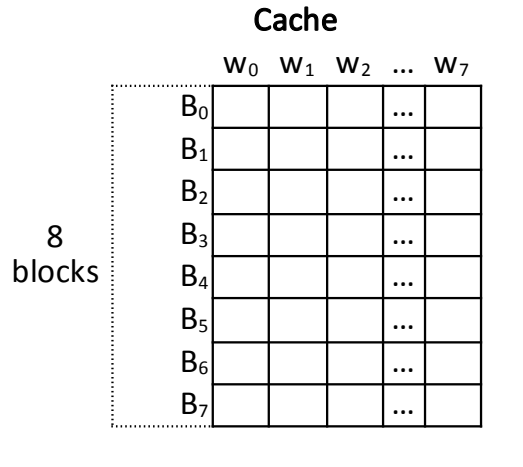
Based on the data given, one can build the following representations:

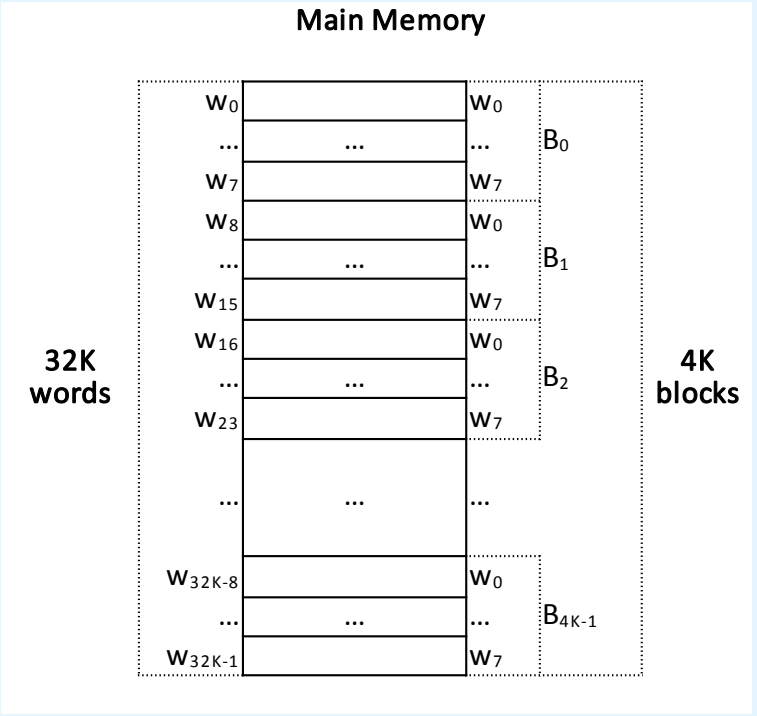
Problem Data:

#BLOCKS(memory) = 4K blocks = 2^2 x 2^10 = 2^12 blocks

#WORDS(memory block) = 8 = 2^3 words

#BLOCKS(cache) = 8 blocks = 2^3 blocks





Consequences:

#WORDS(cache block) = #WORDS(memory block) = 8 words

#WORDS(memory) = #BLOCKS(memory) x #WORDS(memory block)

= 2^12 x 2^3 = 2^15 = 32K words

CACHE3/EAT4 A computer has a main memory of 4K blocks (of 8 words per block) and a direct-mapped cache of 8 blocks.

a) What is the format of a memory address as seen by the cache ?

Solution:

15 bits

tag	block	word
9 bits	3 bits	3 bits
512 groups, of 8 blocks each	8 blocks, of 8 words each	8 words

- cache view: **memory address=<tag, block, word>**
- #BITS(memory address) = log2(#WORDS (memory)) = log2(2^15) = 15 bits
- **#BITS(word)** = log2(#WORDS(cache block)) = log2(2^3) = **3 bits**
- **#BITS(block)** = log2(#BLOCKS(cache)) = log2(2^3) = **3 bits**
- **#BITS(tag)** = #BITS(memory address) – (#BITS(word) + #BITS(block))
  = 15 – (3 + 3) = **9 bits**

CACHE3/EAT4 A computer has a main memory of 4K blocks (of 8 words per block) and a direct-mapped cache of 8 blocks.

b) Compute the hit ratio for a program that loops 2 times from locations 0 to 67 in memory. The cache is initially empty.

Solution:

1st round of the loop

- access to memory word 0 implies a cache MISS and the loading of memory block 0 (with memory words 0 to 7) to the cache block 0
- access to memory words 1 to 7 implies 7 cache HITS
- access to memory words 8 to 63, from memory blocks 1 to 7, has similar implications: 1 MISS and 7 HITS per each block, for a total of 7 MISSES and 49 HITS; cache blocks 1 to 7 get copies of memory blocks 1 to 7
- access to memory word 64 implies a cache MISS, once its hosting memory block (8) also maps to cache block 0 but this cache block holds a copy of memory block 0; memory block 8 (with words 64 to 71) is then copied to cache block 0
- access to memory words 65 to 67 implies 3 HITS
- **at the end of loop 1: #MISSES = 1 + 7 + 1 = 9; #HITS = 7 + 49 + 3 = 59**

CACHE3/EAT4 A computer has a main memory of 4K blocks (of 8 words per block) and a direct-mapped cache of 8 blocks.

b) Compute the hit ratio for a program that loops 2 times from locations 0 to 67 in memory. The cache is initially empty.

Solution (cont.):

2nd round

- access to memory word 0 implies a cache MISS, once its hosting memory block (0) also maps to cache block 0 but this cache block holds a copy of memory block 8; memory block 0 (with words 0 to 7) is then copied to cache block 0
- access to memory words 1 to 7 implies 7 HITS
- access to memory words 8 to 63, from memory blocks 1 to 7, implies 7 x 8 = 56 HITS, once cache blocks 1 to 7 hold copies of those memory blocks
- access to memory word 64 implies a cache MISS, once its hosting memory block (8) also maps to cache block 0 but this cache block holds a copy of memory block 0; memory block 8 (with words 64 to 71) is then copied to cache block 0
- access to memory words 65 to 67 implies 3 HITS
- **at the end of loop 2: #MISSES = 1 + 1 = 2; #HITS = 7 + 56 + 3 = 66**

CACHE3/EAT4 A computer has a main memory of 4K blocks (of 8 words per block) and a direct-mapped cache of 8 blocks.

b) Compute the hit ratio for a program that loops 2 times from locations 0 to 67 in memory. The cache is initially empty.

Solution (cont.):

#ACCESSES= 2 x (67 – 0 + 1) = 2 x 68 = 136
#MISSES = 9 + 2 = 11
#HITS = 59 + 66 = 125

HIT-RATIO = #HITS / #ACCESSES = 125 / 136 = **91,91%**

CACHE3/EAT4 A computer has a main memory of 4K blocks (of 8 words per block) and a direct-mapped cache of 8 blocks.

c) Assume access time for the cache is 22ns and the time required to fill a cache slot from main memory is 300ns. Compute the effective access time in the b) conditions.

Solution:

- cache = level 1 of the hierarchy ; memory = level 2 of the hierarchy
- cache hit-rate = H_1 = 0,9191 (result of question b))
- cache access time = T_1 = 22 ns
- time to fill a cache slot (line) = T_2 = 300ns
  (time spent copying a block from memory into the cache)

overlapped access: $EAT = H_1 \times T_1 + (1 - H_1) \times \mathbf{T_2}$
$= 0{,}9191 \times 22 + (1-0{,}9191) \times 300$
$= 44{,}4902$ ns

sequential access: $EAT = H_1 \times T_1 + (1 - H_1) \times (\mathbf{T_1+T_2})$
$= 0{,}9191 \times 22 + (1-0{,}9191) \times (22 + 300)$
$= 46{,}27$ ns

CACHE4 A computer has a main memory of 2^16 words and a fully associative cache of 64 blocks, with 32 words per cache block.

a) **How many blocks of memory are there?**

b) **What is the format of a memory address as seen by the cache ?**

c) **To which cache block will the memory address $F89C_{16}$ map ?**

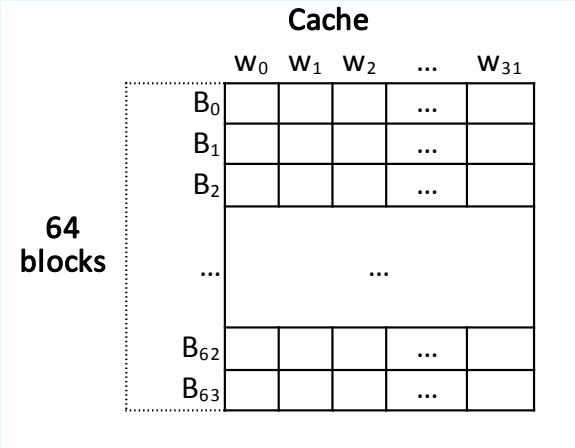
d) **To which memory block, and block word, will the memory address $F89C_{16}$ map ?**

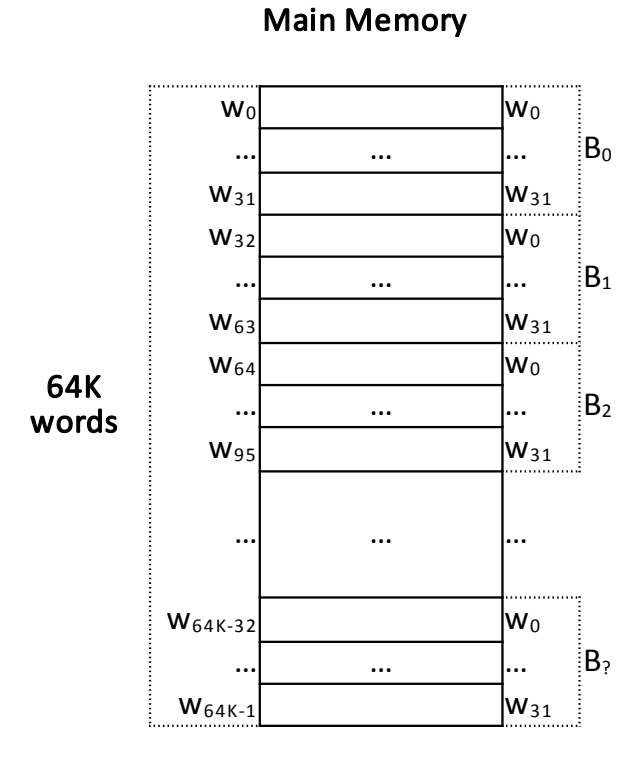
CACHE4 A computer has a main memory of 2^16 words and a fully associative cache of 64 blocks, with 32 words per cache block.

Based on the data given, one can build the following representations:

Problem Data:

#WORDS(memory) = 64K = 2^16 words
#BLOCKS(cache) = 64 = 2^6 blocks
#WORDS(cache block) = 32 = 2^5 words





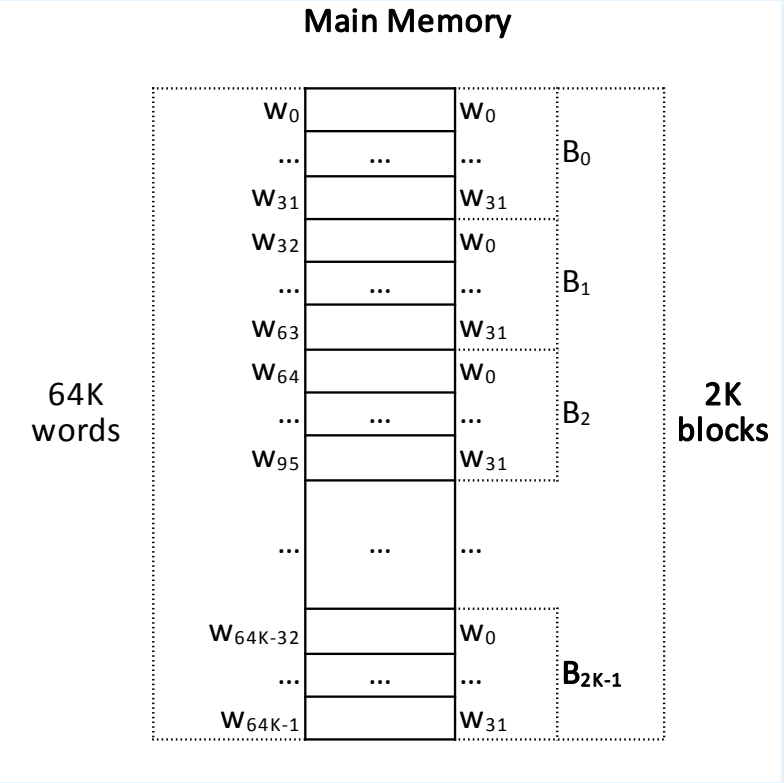
CACHE4 A computer has a main memory of 2^16 words and a fully associative cache of 64 blocks, with 32 words per cache block.

a) How many blocks of memory are there?

Solution:

#WORDS(memory block)
= #WORDS(cache block)
= 32 words

#BLOCKS(memory)
= #WORDS(memory) / #WORDS(memory block)
= 2^16 / 2^5 = 2^11
= 2048 = **2K blocks**



CACHE4 A computer has a main memory of 2^16 words and a fully associative cache of 64 blocks, with 32 words per cache block.

b) What is the format of a memory address as seen by the cache ?

Solution:

- cache view: **memory address=<tag, word>**
- #BITS(memory address) = log2(#WORDS(memory)) = log2(2^16) = 16 bits
- **#BITS(word)** = log2(#WORDS(cache block)) = log2(2^5) = **5 bits**
- **#BITS(tag)** = #BITS(memory address) – #BITS(word)
  = 16 – 5
  = **11 bits**

CACHE4 A computer has a main memory of 2^16 words and a fully associative cache of 64 blocks, with 32 words per cache block.

b) What is the format of a memory address as seen by the cache ?

Solution (cont.):

16 bits

tag	word
11 bits	5 bits
2048 blocks, of 32 words each	32 words

Additional notes:

- the mapping between memory blocks and cache blocks is arbitrary:
  - a memory block may be copied to **any** cache block
- the **tag** field of a cache block states which memory block was copied to that cache block, from **2048** possible candidates

CACHE4 A computer has a main memory of 2^16 words and a fully associative cache of 64 blocks, with 32 words per cache block.

c) To which cache block will the memory address $F89C_{16}$ map ?

Solution:

- Once the cache is associative, then any cache block may receive a copy of the memory block that contains the word having this address !
- Thus, without additional information about the current state of the cache, it is not possible to answer to this question.

CACHE4 A computer has a main memory of 2^16 words and a fully associative cache of 64 blocks, with 32 words per cache block.

d) To which memory block, and block word, will the memory address $F89C_{16}$ map ?

Solution:

- cache view:

$$F89C_{16} = 1111\ 1000\ 1001\ 1100_{2} = 63644_{10}$$

- memory view:

memory address=<memory block, word>

- the **memory block** is given by the bits of the **tag** of the address from the cache view, that is:

$$\textbf{memory block} = 1111\ 1000\ 100_{2} = 1988_{10}$$

- the **word** component is given by the remaining bits:

$$\textbf{word} = 1\ 1100_{2} = 28$$

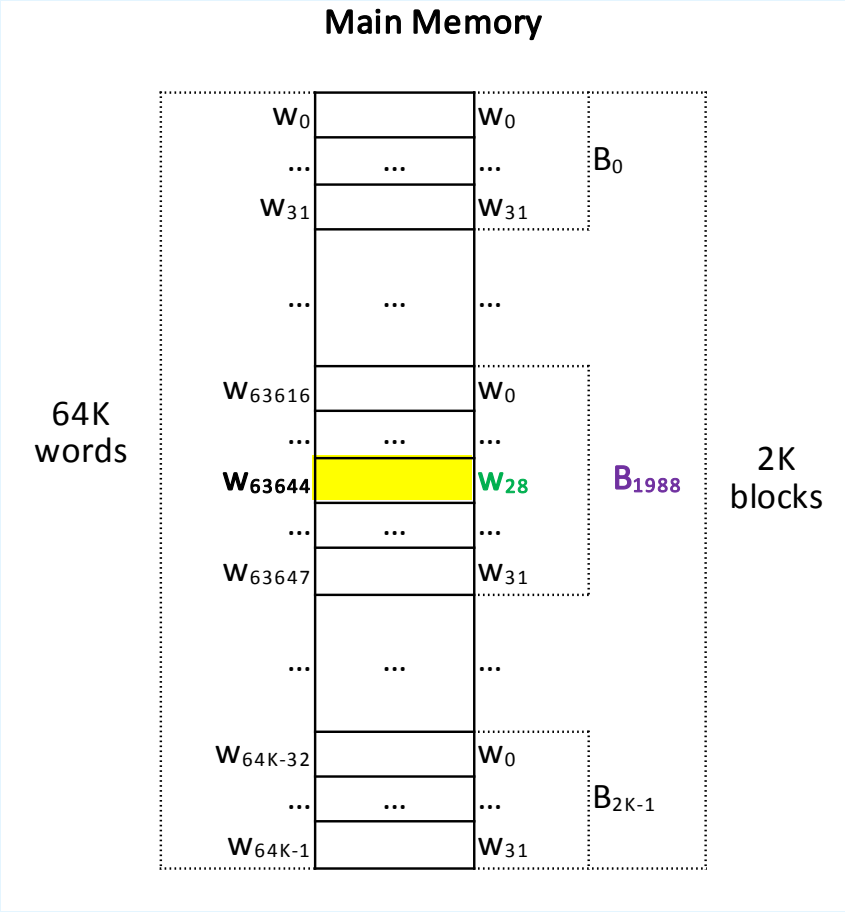
CACHE4 A computer has a main memory of 2^16 words and a fully associative cache of 64 blocks, with 32 words per cache block.

d) To which memory block, and block word, will the memory address $F89C_{16}$ map ?

Solution (cont.):

Notice:

- 63616 = 1988 x 32
- 63644 = 1988 x 32 + 28



CACHE5 A computer has a main memory of 16M words and a fully associative cache of 128 blocks, with 64 words per cache block.

a) **How many blocks of memory are there?**

b) **What is the format of a memory address as seen by the cache ?**

c) **To which cache block will the memory address $01D872_{16}$ map ?**

d) **To which memory block, and block word, will the memory address $01D872_{16}$ map ?**

Homework

CACHE6 A computer has a main memory of 128M words and a 2-way set associative cache of 32K blocks, with 64 words per cache block.

a) **How many blocks of memory are there?**

b) **What is the format of a memory address as seen by the cache ?**

c) **To which cache set, and block of that set, will the memory address $0DB630D_{16}$ map?**

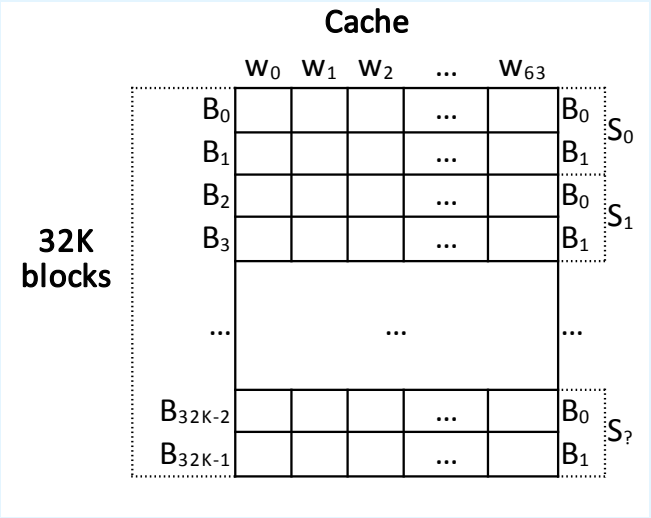
d) **To which memory block, and block word, will the memory address $0DB630D_{16}$ map ?**

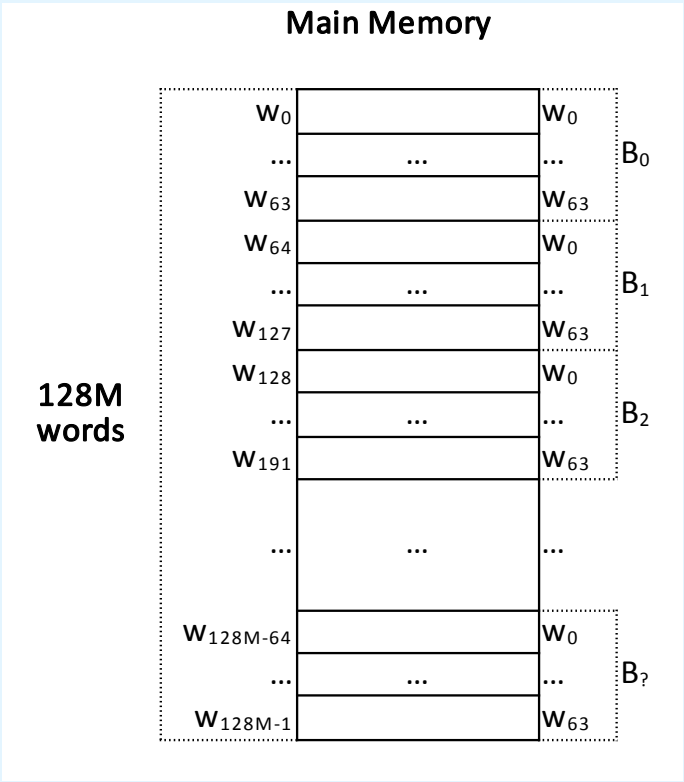
CACHE6 A computer has a main memory of 128M words and a 2-way set associative cache of 32K blocks, with 64 words per cache block.

Based on the data given, one can build the following representations:

Problem Data:

#WORDS (memory) = 128M = 2^7 x 2^20 = 2^27 words
N = number of ways = number of cache blocks per cache set = 2
#BLOCKS(cache) = 32K = 2^5 x 2^10 = 2^15 blocks
#WORDS(cache block) = 64 = 2^6 words





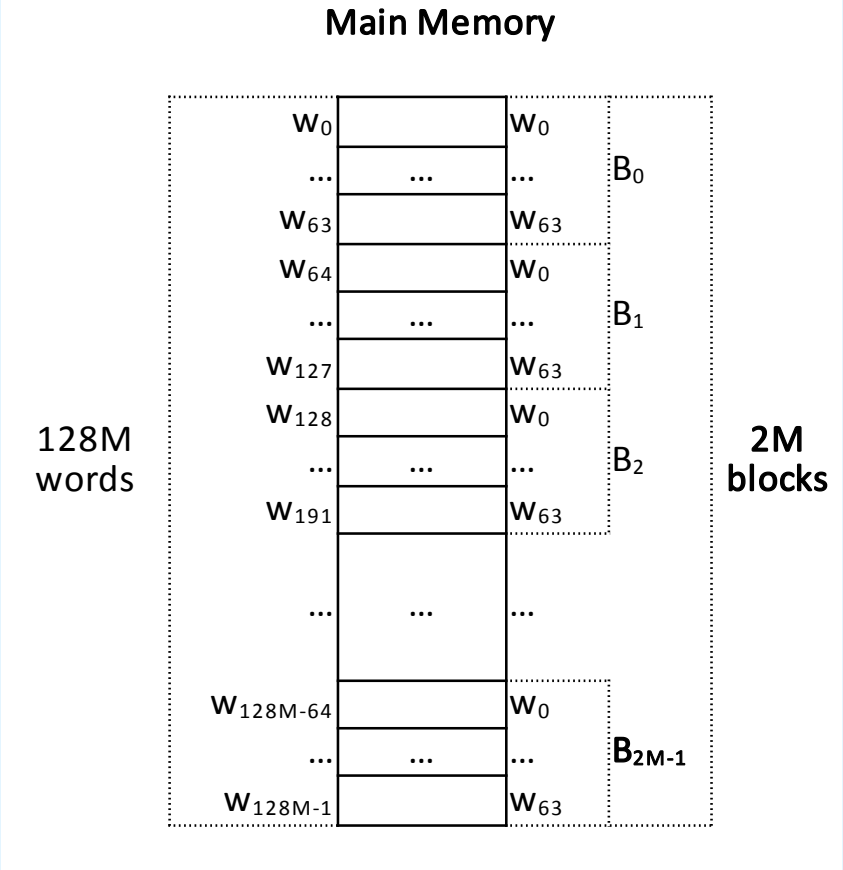
CACHE6 A computer has a main memory of 128M words and a 2-way set associative cache of 32K blocks, with 64 words per cache block.

a) How many blocks of memory are there?

Solution:

#WORDS(memory block)
= #WORDS(cache block)
= 64 words

#BLOCKS(memory)
= #WORDS(memory) / #WORDS(memory block)
= 2^27 / 2^6 = 2^21
= 2^2 x 2^20 = **2M blocks**



CACHE6 A computer has a main memory of 128M words and a 2-way set associative cache of 32K blocks, with 64 words per cache block.

b) What is the format of a memory address as seen by the cache ?

Solution:

- cache view: **memory address=<tag, set, word>**
- #BITS(memory address) = log2(#WORDS(memory)) = log2(2^27) = 27 bits
- **#BITS(word)** = log2(#WORDS(cache block)) = log2(2^6) = **6 bits**
- #SETS(cache) = #BLOCKS(cache) / N = 2^15 / 2^1 = 2^14
- **#BITS(set)** = log2(#SETS(cache)) = log2(2^14) = **14 bits**
- **#BITS(tag)** = #BITS(memory address) – (#BITS(word) + #BITS(set))
  = 27 – (6 + 14) = **7 bits**

CACHE6 A computer has a main memory of 128M words and a 2-way set associative cache of 32K blocks, with 64 words per cache block.

b) What is the format of a memory address as seen by the cache ?

Solution (cont.):

27 bits

tag	set	word
7 bits	14 bits	6 bits
128 candidate blocks, for each set	16K sets (of N=2 blocks each)	64 words

Additional notes:

- a memory block will be copied always to the **same cache set** (that is, memory blocks are directly mapped to a certain cache set)
- a memory block will be copied to **any of the N=2 blocks** of its **cache set** (and in that set, the search for a block is done associatively)
- the **tag** field of a cache block identifies which candidate memory block was copied to that cache block, from **128** possible memory blocks

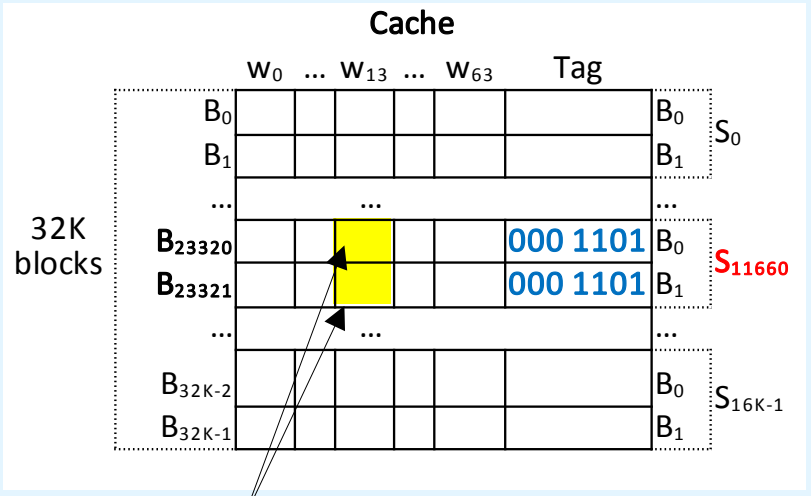
CACHE6 A computer has a main memory of 128M words and a 2-way set associative cache of 32K blocks, with 64 words per cache block.

c) To which cache set, and block of that set, will the memory address $0DB630D_{16}$ map?

Solution: $0DB630D_{16} =$

$0000\ 1101\ 1011\ 0110\ 0011\ 0000\ 1101_{2}$

the bit *0* comes from the conversion of the leftmost hexadecimal digit (*0*), and does not belong to the address, once this one has only 27 bits, and not 28



20 bits		
tag	set	word
7 bits	14 bits	6 bits
000 1101	1011 0110 0011 00	00 1101
	(set 11660)	(word 13)

the block of the set may be any, once the cache is set-associative

CACHE6 A computer has a main memory of 128M words and a 2-way set associative cache of 32K blocks, with 64 words per cache block.

d) To which memory block, and block word, will the memory address $0DB630D_{16}$ map ?

Solution:

- cache view:
  $0DB630D_{16} = 0000\ 1101\ 1011\ 0110\ 0011\ 0000\ 1101_{2} = 14377741_{10}$

- memory view:
  memory address=<memory block, word>

- the **memory block** is found by joining the **tag** and **set** bits of the address from the cache view, that is:
  memory block = $0000\ 1101\ 1011\ 0110\ 0011\ 00_{2} = 224652_{10}$

- the **word** component is given by the remaining bits:
  word = $00\ 1101_{2} = 13_{10}$

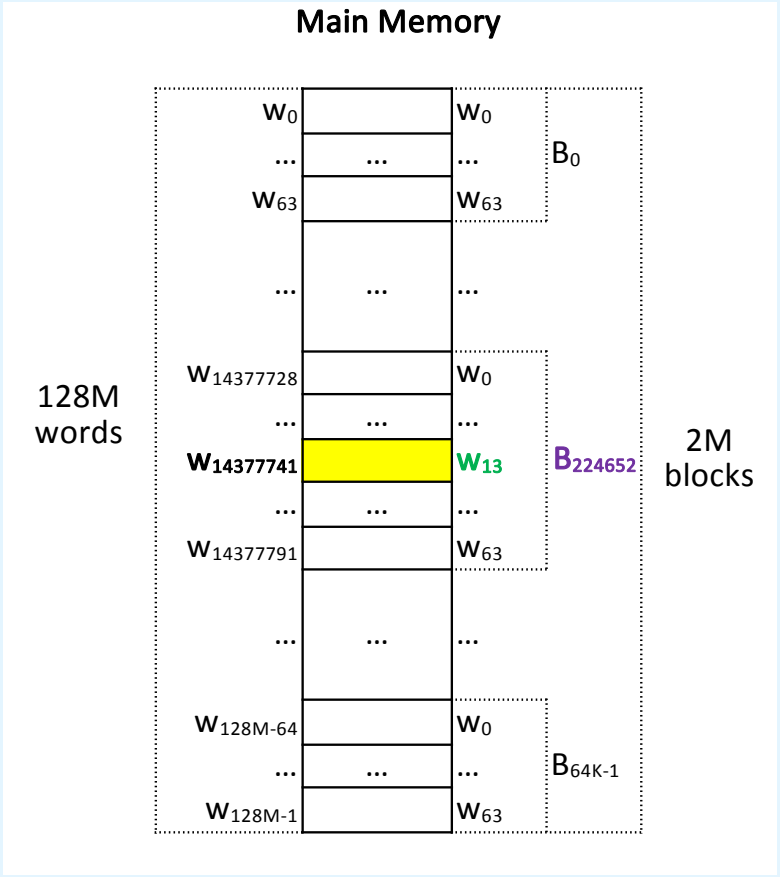
CACHE6 A computer has a main memory of 128M words and a 2-way set associative cache of 32K blocks, with 64 words per cache block.

d) To which memory block, and block word, will the memory address $0DB630D_{16}$ map ?

Solution (cont.):

Notice:

- 14377728 = 224652 x 64
- 14377741 = 224652 x 64 + 13



CACHE7 A computer has a byte-addressable memory with 24-bit addresses, and a 64KB cache with 32 bytes blocks.

Show the format of a memory address as seen by the cache if the cache is:

a) **direct mapped**
b) **(fully) associative**
c) **4-way set associative**

Homework

CACHE8/EAT5 A computer has a main memory of 2K blocks, of 8 words each, and a 2-way set associative cache, with 4 sets of blocks.

a) **What is the format of a memory address as seen by the cache ?**

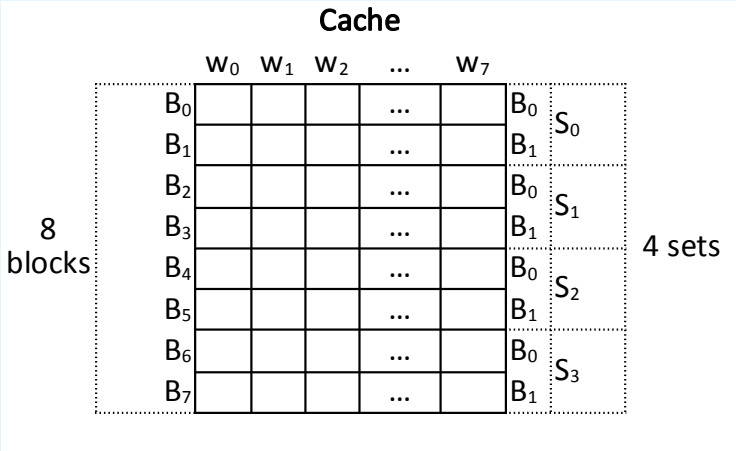
b) **Compute the hit ratio for a program that loops 3 times from locations 8 to 23, and 40 to 47, in memory.**

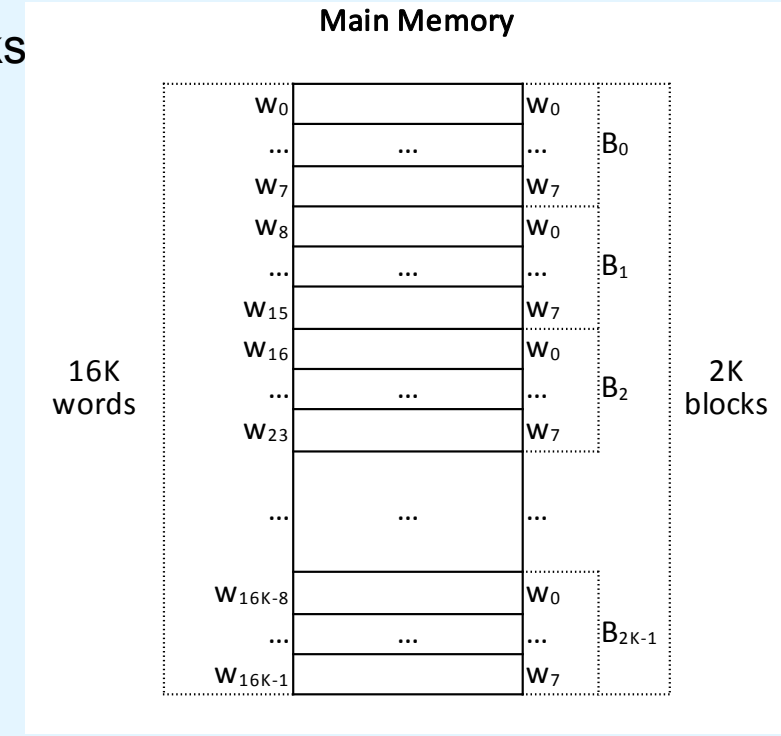
CACHE8/EAT5 A computer has a main memory of 2K blocks, of 8 words each, and a 2-way set associative cache, with 4 sets of blocks.

Based on the data given, one can build the following representations:

Problem Data:

#BLOCKS(memory) = 2K = 2^1 x 2^10 = 2^11 blocks
#WORDS(memory block) = 8 = 2^3 words
N = num. of ways = num. of blocks per set = 2
#SETS(cache) = 4 = 2^2 sets





Consequences:

#BLOCKS(cache) = #SETS(cache) x N = 4 x 2 = 8 = 2^3 blocks
#WORDS(memory) = #BLOCKS(memory) x #WORDS(memory block)
= 2^11 x 2^3 = 2^14 = 16K words

CACHE8/EAT5 A computer has a main memory of 2K blocks, of 8 words each, and a 2-way set associative cache, with 4 sets of blocks.

a) What is the format of a memory address as seen by the cache ?

Solution:

14 bits

tag	set	word
9 bits	2 bits	3 bits
512 candidate blocks, for each set	4 sets (of N=2 blocks each)	8 words

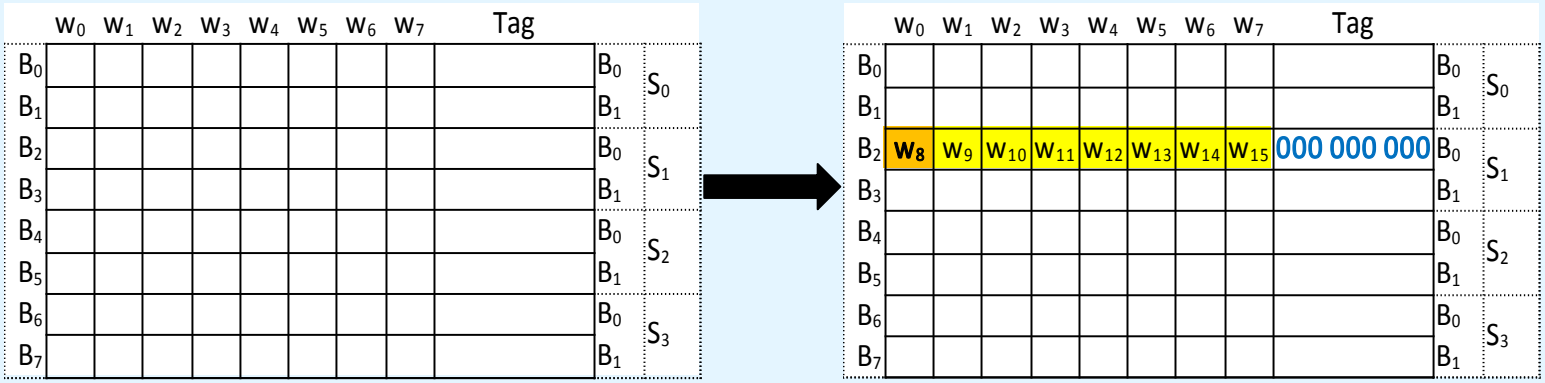
- cache view: **memory address=<tag, set, word>**
- #BITS(memory address) = log2(#WORDS(memory)) = log2(2^14) = 14 bits
- **#BITS(word)** = log2(#WORDS(cache block)) = log2(2^3) = **3 bits**
- **#BITS(set)** = log2(#SETS(cache)) = log2(2^2) = **2 bits**
- **#BITS(tag)** = #BITS(memory address) – (#BITS(word) + #BITS(set))
  = 14 – (3 + 2) = **9 bits**

CACHE8/EAT5 A computer has a main memory of 2K blocks, of 8 words each, and a 2-way set associative cache, with 4 sets of blocks.

b) Compute the hit ratio for a program that loops 3 times from locations 8 to 23, and 40 to 47, in memory.

Solution:

1st round:



8_{10} = 000000000 01 000$_2$ => tag=0, set=1, word=0 => **MISS** and block=0

9_{10} = 000000000 01 001$_2$ => tag=0, set=1, word=1 => HIT in block 0

…

15_{10} = 000000000 01 111$_2$ => tag=0, set=1, word=7 => HIT in block 0

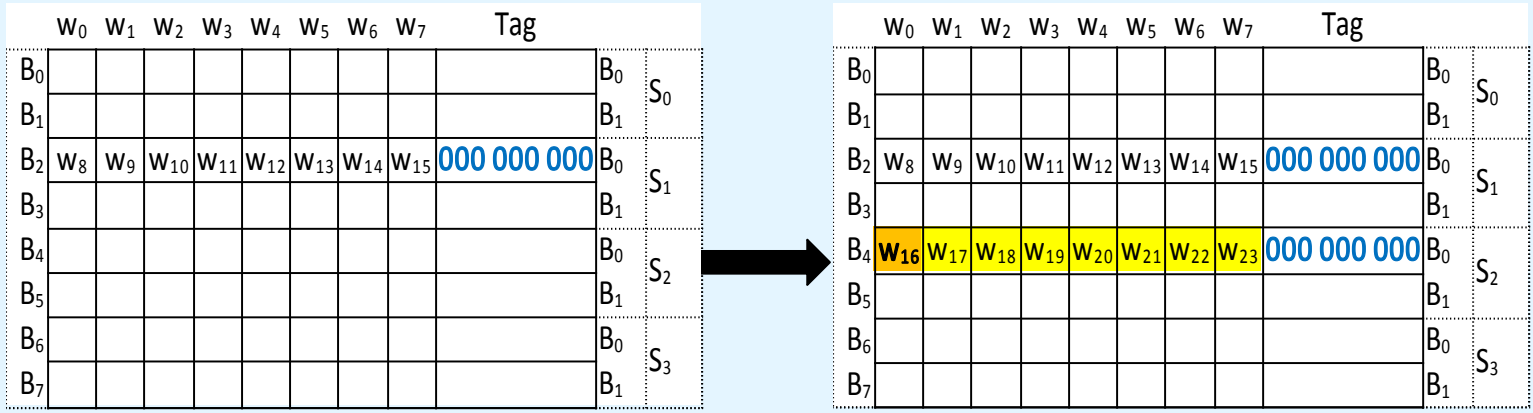
7 HITS

CACHE8/EAT5 A computer has a main memory of 2K blocks, of 8 words each, and a 2-way set associative cache, with 4 sets of blocks.

b) Compute the hit ratio for a program that loops 3 times from locations 8 to 23, and 40 to 47, in memory.

Solution:

1st round (cont.):



16_{10} = 000000000 10 000$_2$ => tag=0, set=2, word=0 => **MISS** and block=0

17_{10} = 000000000 10 001$_2$ => tag=0, set=2, word=1 => HIT in block 0

…

23_{10} = 000000000 10 111$_2$ => tag=0, set=2, word=7 => HIT in block 0

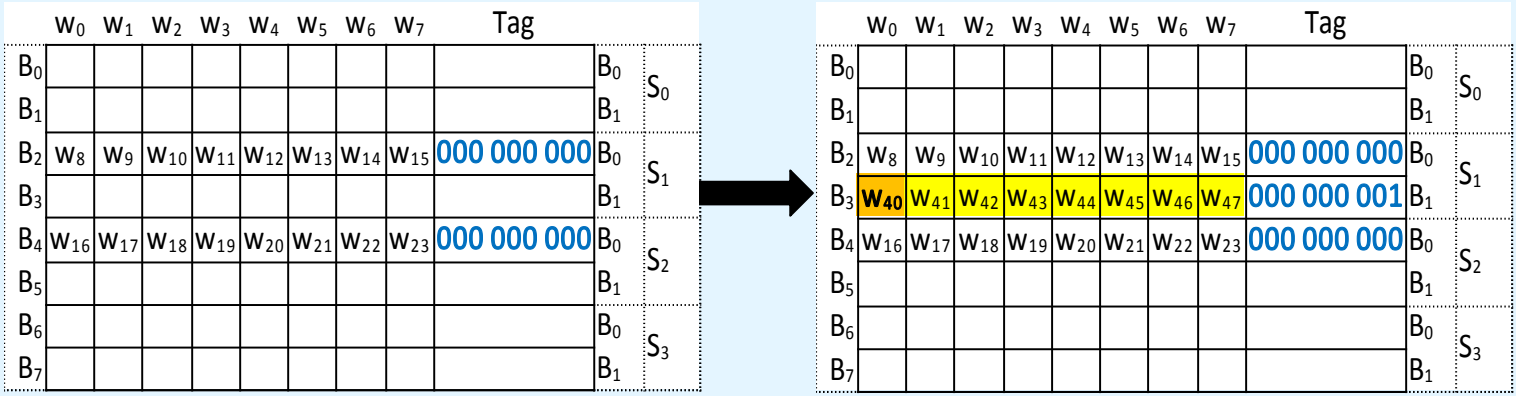
7 HITS

CACHE8/EAT5 A computer has a main memory of 2K blocks, of 8 words each, and a 2-way set associative cache, with 4 sets of blocks.

b) Compute the hit ratio for a program that loops 3 times from locations 8 to 23, and 40 to 47, in memory.

Solution:

1st round (cont.):



40_{10} = 000000001 01 000$_2$ => tag=1, set=1, word=0 => **MISS** and block=1

41_{10} = 000000001 01 001$_2$ => tag=1, set=1, word=1 => HIT in block 1

…

47_{10} = 000000001 01 111$_2$ => tag=1, set=1, word=7 => HIT in block 1

7 HITS

CACHE8/EAT5 A computer has a main memory of 2K blocks, of 8 words each, and a 2-way set associative cache, with 4 sets of blocks.

b) Compute the hit ratio for a program that loops 3 times from locations 8 to 23, and 40 to 47, in memory.

Solution:

<u>2nd and 3rd rounds:</u>

	w_0	w_1	w_2	w_3	w_4	w_5	w_6	w_7	Tag		
B_0										B_0	S_0
B_1										B_1	
B_2	w_8	w_9	w_{10}	w_{11}	w_{12}	w_{13}	w_{14}	w_{15}	000 000 000	B_0	S_1
B_3	w_{40}	w_{41}	w_{42}	w_{43}	w_{44}	w_{45}	w_{46}	w_{47}	000 000 001	B_1	
B_4	w_{16}	w_{17}	w_{18}	w_{19}	w_{20}	w_{21}	w_{22}	w_{23}	000 000 000	B_0	S_2
B_5										B_1	
B_6										B_0	S_3
B_7										B_1	

8_{10} = 000000000 01 000_2 => tag=0, set=1, word=0 => HIT in block 0

…

15_{10} = 000000000 01 111_2 => tag=0, set=1, word=7 => HIT in block 0

8 HITS

16_{10} = 000000000 10 000_2 => tag=0, set=2, word=0 => HIT in block 0

…

23_{10} = 000000000 10 111_2 => tag=0, set=2, word=7 => HIT in block 0

8 HITS

40_{10} = 000000001 01 000_2 => tag=1, set=1, word=0 => HIT in block 1

…

47_{10} = 000000001 01 111_2 => tag=1, set=1, word=7 => HIT in block 1

8 HITS

CACHE8/EAT5 A computer has a main memory of 2K blocks, of 8 words each, and a 2-way set associative cache, with 4 sets of blocks.

b) Compute the hit ratio for a program that loops 3 times from locations 8 to 23, and 40 to 47, in memory.

Solution: (cont.):

#ACCESSES= 3 x [(23-8+1)+(47-40+1)] = 3 x 24 = 72
#MISSES = 3
#HITS = (7+7+7) + 2 x (8+8+8) = 21 + 48 = 69

HIT-RATIO = #HITS / #ACCESSES = 69 / 72 = **95,83%**